Java Script notes

Es6 introduced concepts:

Let, const, template string, default parameterized function, arrow functions, Promises

The official documentation available in **web mdn JavaScript**

- JavaScript is the high-level Object oriented and multiple paradigms and it is a programming language.
- JavaScript is a programming language that allows developers to create the interactive pages and it is core technology of world wide web.

Uses

It is used to refresh the social media feeds and create the animations, create interactive maps, create click to show dropdown menus and change element colours in the webpage.

1. JavaScript is a versatile and beginner friendly language.
2. It is only a single language that your browser will understand.

Three Tier Architecture of the Application:

- There tier application is a model that divides an application into three inter connected layers.
  1. **Presentation layer:**
     The user interface where the end user interacts with the system.
     Ex: Web browser or mobile application.
  2. **Logic layer or Business Layer:**
     This is the middle tier or layer of the architecture also known as Business layer.
     It handles the application core processing business rules and calculations.
     Ex: Java, Python, JavaScript
  3. **Database Layer:**
     This layer manages the storage or retrieval and manipulation of the application data, typically utilizing the database.

User Interaction-:

- The user interacts with the prestation tier for example enter the data in the web form or clicks the button.

Request Processing: -

- The presentation tier sends the user request to the application layer or tier

Business Logic: -

- The logic tier executes the relevant business logics, process the data and potentially interacts with the database or data layer to retrieve or store the data.

Data Access:

- If necessary, the application tier communicates with the data tier to access the database.

JAVASCRIPT

Response:

- The logic tier formulates a response based on the process data and business rules and packages it into the expected format that your presentation layer required.

Display:

- The presentation receives the response from the application tier and displays the information to the user.

JavaScript was developed by **Branden Eich**

What is a JavaScript?

- JavaScript is high level programming language which is used to create interactive webpages
- It is the only language understood by the browser
- JavaScript is scripting language.
- It is a language you can use at a browser side as well as server side.
- It is most commonly and popular language used right now.
- Lot of framework and libraries based on JavaScript it can be used for both frontend and backend.
    - Ex: In frontend we use reactJs, angular js, nextJs.
    - For backend we use NodeJS, expressJs.

History of the JavaScript:

In 19995 JavaScript is created by Brendin Eich Netscape corporate communication.

Mocha ---------- Live script ---------- JavaScript ---------- ECMA Script

Net Scape Communication Corporation

Brenden Eich (1995)

Mocha

Live Script

Sun Microsoft systems

JavaScript

ECMA (European computer manufacture corporation)

Characteristics of the JavaScript:

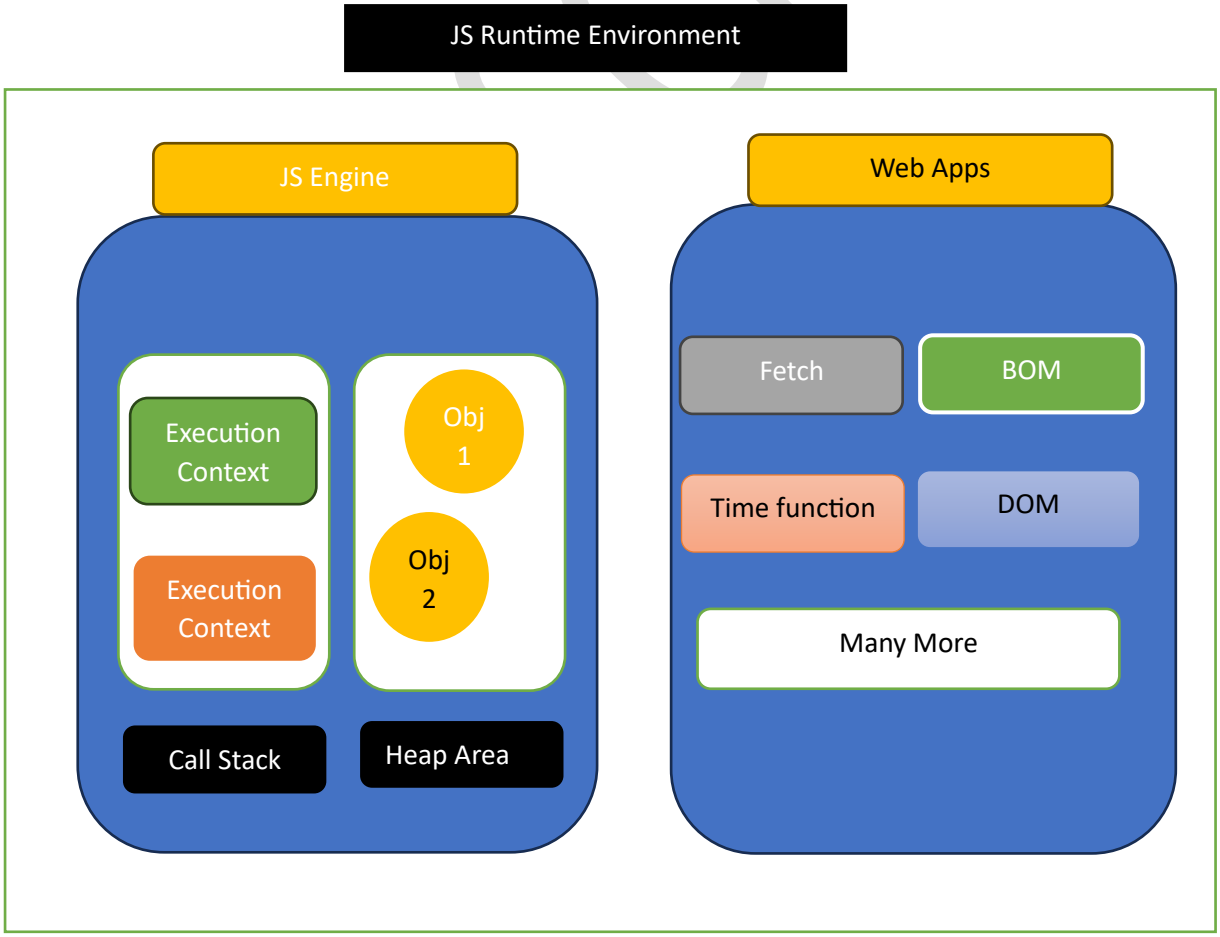
1. JavaScript is interpreted language.
2. Client-side Scripting language.
3. High level programming Language.
4. JavaScript is loosely (or) weekly typed language.
5. JavaScript is dynamically typed language.
6. It is synchronous and single threaded language.
7. It is Object based language.
8. After ES6 it is also called object-oriented programming language.
9. It is light weight language.

Object oriented Programming language:

- Typically, it uses classes, supports inheritance, polymorphism, Encapsulation, Abstraction.

Object base language:

May use prototypes or structures and often focus on practical use of inbuilt objects like math, date rather than strict adherence of object-oriented principles.



Working of JS Engine:

1. JavaScript Engine is a simple computer program that interprets JavaScript code.
2. The JS Engine is responsible for executing the code.
3. The Engine interprets and compiles (JIT compiler) JavaScript into machine code so it can be executed by CPU
4. Any JavaScript engine typically contains a call stack and heap area.
5. The call stack is where the code is executed.
6. The heap area is unstructured memory pool that store all the objects needed for the application.

Parser:

- This is the first stage of the engine every time we run a JavaScript program our code is received by the parser inside the JS engine
- The parser job is to check the syntactic error in line-by-line manner and convert it into the AST format (Abstract Structure Tree).

AST (Abstract Structure Tree) or (Abstract syntax tree):

- Once the parser checks all the JavaScript code and get satisfied that there is no mistakes or error in the code then it creates the data Abstract structure tree.

JIT Compiler:

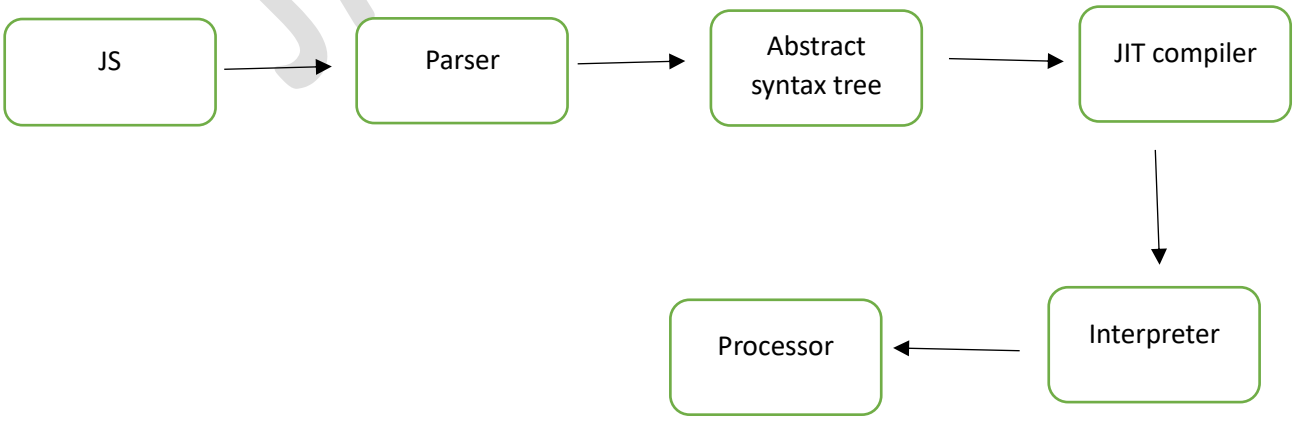
- With the help of the Jit compiler we can convert into machine code language, once convert it is given to the interpreter.

Interpreter:

- It executes the code given by the Jit compiler in line-by-line manner.

Processor:

- The processor role in the JavaScript engine involves executing instruction generated by the Engine whether they are interpreted byte code or Jit compile machine code.



JAVASCRIPT

Uses of JavaScript:

1. Web application
2. Web development
3. Mobile
4. Game developments
5. Presentation and slide shoe
6. Server application
7. Web servers
8. Client-side validations
9. Display pop-up windows and dialogue box
10. Animate elements
11. Dynamic Drop-down menu

Difference between java and JavaScript

Java	JavaScript
Java is programming language	JavaScript is scripting language
It is multi-threaded language	It is single thread language
Java is strictly typed language	It is loosely typed
It runs on JVM	It runs on all the browser ex: chrome, brave
More Memory use in java	It uses less memory
It is independent language	Dependent on html

Browser	Js Engine
Chrome	V8 Engine
Fire fox	Spider monkey
Safari	JavaScript core
Internet Explorer	Chakra
Brave	V8+blink

Heap Memory:

- This is the place where all the object are stored which are necessary in the application.

Call Stack:

- It is place where your JavaScript code get executed.

Execution Context:

- When the JavaScript code it executes within an execution context this context includes the global context for code outside of the functions and function context for code inside the functions.
- Each context has its own scope variable and functions.

JAVASCRIPT

Ways to add a JavaScript and to execute:

1. We can execute JavaScript instructions in the console provided by the browser.
2. We can execute JS by embedding in the html page.

   Internal way of adding JavaScript:

   - With the help of <script> tag we can embedded the java script code.

```
ch-ATT > JavaScript > index.html > html
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Document</title>
    <script src="./index.js"></script>
</head>
<body>
    <!-- Internal way of adding javascript  -->
    <script>
        console.log("Internal way of adding javascipt");
    </script>

</body>
</html>
```

We wont use script tag in head section as when executing the html file initially head will execute along with it js code also get executes which may cause some errors.

Defer:

It specifies that the script is download in parallel to parsing the page and the script executes after the page has finished parsing.

If the user wants to write the script in the head, then the user needs to write defer so that the after

Note:

The defer attribute only for external scripts (should only be used only if src attribute present.

Ex: <script scr=" ./index.js" defer></script>

b. External way of adding a JavaScript:

- we can create a separate file for JavaScript code with extension **.js**.
- Link the js file with html page using src attribute in side the opening of script tag.

Ex: index.html

```html
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Document</title>
    <script src="./index.js"></script>
</head>
<body>
</body>
</html>
```

Index.js

```
PEbatch-A11 > Java  D:\Web Tech NEw\Tailwind css\divcenter.html
1   console.log("Using the external");
```

Tokens:

Tokens is a smallest unit of any programming language, there are various types of tokens in a JavaScript.

1. Keyword
2. Identifier
3. Literal
4. Operator
5. Separator
6. Comment

1. **Keywords:**
   keywords are the predefined words which haves some special meaning.
   Keywords are always in lower case
   It is understood by JavaScript.
   Ex: var, let, const, async, break, continue, function.
2. **Identifiers:**
   - Identifier is nothing but the name provided to any variable, class, function or object.
     Rules of identifiers:
     1. You can't use keywords as identifier.
     2. Identifier name start with a number can have number between it.
     3. It cannot have any special character expect underscore and $
     4. It does not contain any space in between.
3. **Literal:**
   - Literal is nothing but the data provided by the user.

4. **Separator:**
    - Separator are used to differentiate the statement written in a JavaScript file.
    - Ex: semi-colon, single quotes, double quotes, comma.
5. **Comments:**
    - Comments are the line of code that are not executed by the browser.
    - Comments are used to add the notes, description and explanation about the JS code.
    - JavaScript support 2 types of comments.
        1. Inline comment (//)
        2. Multi line comment (/* Dhanush*/)
6. **Operators:**
    - Operators are the predefined symbols which are performing some specific tasks.
    - Ex: x+y=z, x, y are operands and + is an operator and z is the result.
    - **Types of Operators:**
        1. Arithmetic Operator: - -→ Perform arithmetic operations.
        2. Assignment operator:
        3. Relational or comparison operator: It compare the values and datatypes.
        4. Logical Operator: It combines expression and make the decision.
        5. Conditional operator: Evaluate true or false based on the condition.

Variables:

- Variable is nothing but name given to the block of memory.
- We can create a variable in java script using variable declaration followed by variable name.
- <u>Syntax:</u> variable_declaration variable_name=value;
- There are 3 types of variable declaration
    1. Var
    2. Let (added in ES6)
    3. Const (added in ES6)
- Var A; →Declaration of a variable
- A=20; →Initialisation of a variable
- Var b = 30; → declaration and initialisation of variable
- A=40; → reinitialization
- Var A = 50; →Re declaration

1. **<u>Var:</u>**
    - Var is the traditional way of declaring the variables.
    - The var statement declare function scope or global scope variables optionally initializing the value to the variable.
    - It can be redeclare and updated within its scope.
    - It can be declared without initialization.
    - It can be accessed without initialization as its default value as undefined.

JAVASCRIPT

- These variables are hoisted. [when ever we are trying to access the value be before declaration or initialization that process is known as hoisting where it shows undefined.]

What is the hoisting? **

- It is a processing of accessing the variables before its initialization this will be possible only if the variable is declared with the **var** type variable declaration.

What is the Temporal Dead Zone? **

- If any variable is declared with let or constant and trying to access that variable before its initialization then the variable present in the temporal dead zone.
- And it returns an uncaught reference error.
- Def: It is a time interval between start of block and the point where a variable is declared during this time the variable exists but cannot be accessed or used.
- Temporal Dead zone s only applied to variables declared with let and const.

PEbatch-A11 > JavaScipt > variables.js > ...

```javascript
//var type declaraion
console.log(a) // Undefined because of hoisting and it possible with var keyword
var a;//decalaration
console.log(a); //Undefined

a = 20;//intialisation
console.log(a) //10

var a = 60; //re declaration
console.log(a) //60

console.log(c)  //uncaught Reference error Because of temporal dead zone
let c;
```

Bare Declaration: **

Whenever we are declaring a variable without using var, let, const declarations then JS engine automatically treats the declaration as Bare Declaration.

Bare declaration works only when the variable is not in any block or function i.e. it works only when the variable is declared globally.

2. **Let:**
   - The scope of let variable is the block scope.
   - It can be updated but cannot be redeclared in the same scope.
   - It can be declared without initialization.
   - It cannot be accessed before initialization.
   - If you are trying to access after the declaration without initialization, we will get value as the undefined.

JAVASCIPT

```
let c;
console.log(c)//undefined

c=30
console.log(c)

c=50
console.log(c)

let c="Dhanush" // We cannot redeclare the varibale
                //Uncaught SyntaxError: Identifier 'c' has already been declared
```

3. **Const**:
   - The scope of the const variable is block scope.
   - It can neither be updated or redeclared in any scope.
   - It cannot be declared without an initialization.
   - It cannot be accessed without initialization.

```
const a; // Declaration is not possible with const type declaration error

var console: Console
console.log(b) //10

b=20;
console.log(b) //Error re initialization is not possible in js
```

	Declaration	Initialization	Declaration + initialization	Re-declaration	Re-initialization	Redeclaration+ Re initialization
var	YES	YES	YES	YES	YES	YES
Let	YES	YES	YES	NO	YES	NO
const	NO	NO	YES	NO	NO	NO

Difference Between var, let, const:

var	let	const
Var is a global scope	Block scope	Block scope
We can declare multiple variables with same name (most recently created variable will be used)	We cannot declare more than one variable with same name in a block scope.	We cannot declare more than one variable with same name in a block scope.
We can declare var without initialization E.g.: var a;	We can declare let without initialization e.g. let a;	We cannot declare const without initialization
A variable declared using var belongs to global scope(window) we can access using window object.	A variable declared using let does not belong to global scope so we cannot use window object	The variable declared using const does not belong to global scope so we cannot use window object.

The variable declared using var is hoisted, that means it can be used before initialization.	The variable declared using let is not hoisted that means it cannot be used before initialization.	The variable declared using const is not hoisted that means it cannot be used before initialization.

```
var age=22;
console.log(22)
console.log(window) // age is present inside the window object
//anything which is present inside the window object is in the global scope
console.log(window.age)

// function demo()
// {
//     var b = 20; //! Funciton Scope
//     console.log(b);
// }
// console.log(window)
// console.log(b) //! uncaught ReferenceError: b is not defined
//                //! bcoz b is present inside the function scope

// {
//     var myName="Dhanush" //! it is present in global scope
//     console.log(myName) // Dhanush
// }
// console.log(myName)//Dhanush


let a=20;
console.log(window.a)// undefined //!it is present in script scope

{
    let b=10;
    console.log(b) // 10 //! it is present in scope
}
// console.log(b)//! Error variable_scope.js:30 Uncaught ReferenceError: b is not defined
```

```
function demo(){
    let a=20;
    const c=30;
    console.log(a,c)
}

demo()
console.log(a,c)
```

```
Elements  Console  Sources  »   1
top ▾   Filter   Default levels ▾   No Issues

20 30                                   variable_scope.js:48
▸Uncaught ReferenceError: a is not      variable_scope.js:52
defined
    at variable_scope.js:52:13
>
```

Scopes:

- scope is the area where a variable exist and accessable

1. **Block Scope:**
    - Whenever we are declaring a variable with let, or const inside the curly braces is known as block scope.
    - Those variables that are declared inside the function have local or function scope which means that we cannot access outside of the function.
2. **Function Scope:**
    - Whenever we are declaring a variable with var, const or let then that particular variable is under function scope.
    - Those variables that are declared inside the function have local or function scope which means that we cannot access outside of the function

3. **Script Scope:**
   - When we are declaring a variable with let or const outside the function and curly braces then that particular variable is under script scope.
4. **Global Scope:**
   - Whenever we are declaring a variable with var type declaration outside of the function and this variable is stored inside the window object then we are calling this variable under the global scope.
   - Those variables which are declare outside of the function are inside the global scope.
   - In JavaScript global variable can be accessed from anywhere.

Data Types:

- Data types are used to define what type of data we are going to store in a particular variable.
- Data types are used to specify which type of value a variable can hold.
- They define the kind of data present inside the variable.
- JavaScript provides different data types to store the different types of values.
- There are 2 types of datatypes present in JS.

What is the dynamically typed language? **

- JavaScript is a dynamically typed language which means you do not need to specify the type of data that particular variable can hold. The type is determined at runtime based on the value assigned to the variable

1. Primitive Data Types

- **In** JavaScript primitive datatype is the data that is not an object and has no methods or properties.
- The primitive datatypes are single value data
- There are 7 primitive datatypes
  i) Boolean
  ii) String
  iii) Number
  iv) Undefined
  v) Symbol
  vi) Null
  vii) Big int (introduced in ES6)
- <u>NOTE:</u> All primitive types expect null and undefined have their corresponding object wrapper types.
  - Number:
    - Number datatype can use decimal as well as non-decimal values.
    - Ex var a = 10;
      Let myage=22;

JAVASCRIPT

```
//! number data type
var a=10;
console.log(a)
console.log(typeof a) // number [tells which type of datatype it is]

let b=10.4
console.log(b, typeof b) // number

//! typeof: it is a operator which is used check which
//!         type of data we store inside a variable
```

- String:
  - The string datatype in the JavaScript represents a sequence of characters that are surrounded by single or double quotes and template literals (`Dhanush`)
  - Typically, it is used to represent the text.
  - Ex: var fname=" Dhanush"
  - Var lname = 'Pagolu'
  - Var fullname=`full name is ${fname} ${lname}`

```
//? string data type
var fname="Dhanush"
console.log(fname)          //Dhanuhs
console.log(typeof fname)   //string

var lname = 'pagolu'
console.log(lname)          //pagolu
console.log(typeof lname)   //string

var fullname=`My full name is ${fname} ${lname}`
console.log(fullname)       //My full name is Dhanush pagolu
```

- **Boolean**

  1. Java script Boolean represent true or false values
  2. It is used for logical operations, conditional testing and variable assignments based on conditions.
  3. values like 0, Nan (not a number), Empty string (""), undefined are the falsie values.
  4. Non empty strings other than 0, objects and array are truthy values.

  Note: A falsie value is a value that is considered as false when encountered in Boolean context.

```
var ismarried=false
console.log(ismarried)          //false
console.log(typeof ismarried)   //boolean

console.log(false==0)           //true
console.log(false === 0)        //false
console.log('1' == 1)           //true
console.log('1' === 1)          //false
console.log(1+true)             //2
console.log('1'+true)           //1true
console.log("1"-true)           //0
console.log(0+undefined)        //Nan
console.log(true-"aa")          //Nan
```

- **Undefined:**

1. This means that a variable has been declare but has not been assigned a value or it has been explicitly set to the value undefined.

2. Let a = undefined
3. Console.log(a); //undefined

```
//! undefined data type
let x;
console.log(x)                    //undefined
```

- **Big Int:**
  1. In JavaScript big int is the numeric datatype that can represent integers in the arbitrary precession format. [no limit in range of numbers]
  2. Big int value is also known as big int primitive value which is created by appending 'n' to an integer literal.

```
//! Big Int     2^53
let n=1231343424534534553635n;
console.log(n)                  //1231343424534534553635n
console.log(typeof n)           //bigint

let c=123134342453453553635
console.log(c)                  //1.2313434245345347e+21 //! we should keep n
console.log(typeof c)           //number
let d=122424234n+345
console.log(d)                  //! Uncaught TypeError: cannot mix BigInt
```

- **Null:**
  1. Null is an empty value.
  2. Null is not same as the zero.
  3. Null is the absence of any value.
  4. Ex: console.log (null == undefined) //true
  5. Ex: console.log (null === undefined) // false

```
//! Null datatype
let balance=null
console.log(balance)            //null
console.log(typeof balance)     //Object
```

Difference between null and Undefined? **

<u>Type Coercion:</u>

- Type coercion is also known as implicit type conversion.
- **Implicit conversion:**

- In the java script values of different datatypes automatically converted to a common type before an operation is performed.

- This can be sometimes led to unexpected result or expected results if not understood properly.
- Type coercion scenarios:

i) Arithmetic operations:

- Combination of numbers and string:

```
let a=10;
let b='11';
let c=a+b;  //string
console.log(c); // '1011'
-------------------------------------
let x=" true"
console.log(x+5);  //'true5'
-------------------------------------
let u=12
console.log ("Dhanush" +u)        //" Dhanush12"
```

ii) Comparison Operator:

```
Ex: console.log (10 == '10') // true
    Console.log (10 === '10')       //false
    Console.log (false == 0)        //true
```

iii) Strict equality operator (===): Avoiding coercion to avoid the unexpected behaviour from type coercion use the strict equality operator.
It checks both value and datatype.

```
Console.log (10 === '10')  //false
Console.log (10 === 10)    //true
```

```
console.log(10&&5)  //5
console.log(3&&5)   //5
console.log(5&&3)   //3
console.log(7||4)   //7
console.log(0&&4)   //0
console.log(0||9)   //9
console.log(0||1)   //1
console.log(undefined && null)  //undefined
console.log(null || undefined)  //undefined
console.log(NaN && null)        //NaN
console.log(null && NaN)        //null
console.log(!(undefined && null)) //true
console.log(null || 56454)      //56454
console.log(undefined && 7858)  //undefined
console.log(!(undefined&&null) && (NaN&&null))  //NaN
```

Falsie values in JavaScript: Null, undefined, 0, NaN

JAVASCRIPT

Type conversion:

- In JavaScript type conversion is the process of changing the value of one data type to another datatype.
- It is essential for performing operations and comparisons between different datatypes
- **Explicit Conversion:**
  You can manually convert datatypes using built in functions like Number (), String (), Boolean ().
- **Common Explicit type conversion:**
  Scenarios:
  1) Number to String:
     Ex: here we are converting the number datatype into string by using String () built in function:
     Let a = 1001
     Let d=String(a)
     Console.log(d) //"1001"
     Console.log (typeof d) //string
  2) String to Number:
     Ex: here we are manually converting the string datatype to number
     Let a = "1001"
     Let b=Number(a)
     Console.log(b) //1001

```
let doy='2002'
console.log(doy)
console.log(typeof doy)

//! string to number conversion

let c=Number(doy)
console.log(c)
console.log(typeof c)

let a='Dhanush'
let d=Number(a)
console.log(d)//Nan (not a number)

//! number to string conversion
let b=3000;
let e=String(b)
console.log(typeof e)
```

JAVASCRIPT

```
//! boolean to number
let f=true
let f1=Number(f)
console.log(typeof f1, f1)  //number 1
//! number to boolean

let first=10
let boolvalue=Boolean(first)
console.log(typeof boolvalue, boolvalue)    //boolean true

let last=null
let boolval=Boolean(last)
console.log(typeof boolval, boolval)        // boolean false
```

Where all the default CSS present? **

Ans: In the user agent stylesheet

Decision Making Statements

1.) If statement:

It is used for some statement has to be executed based on the single condition.

Syntax:

```
if(condition)
{
        //statements
}
```

Ex: let cookiesAvaliable=1

```
if (cookiesAvaliable == 1)
{
    Console.log ("I will eat one cookie");
}
If (cookiesAvaliable == 2) // error
```

2.) If else statement

- It is use when some set of code has to be executed based on some condition otherwise some other set of code is executed.

- If statement tells us that if a condition is true, it will execute a block of a statement and if the condition is false else block get executed.
- Syntax:

```
if(condtion)
{
    //set of stmts //! when condition is true it will exceute
}
else
{
    //set of stmts //! condition is false it will
}
```

Example:

```
const a=prompt("Enter a value")
const b=prompt("Enter b value")
if(a<b)
{
    alert("a is smaller")
}
else{
    console.log("b is smaller")
}
```

3.) Else if ladder:

- It is used when some set of statements has to be executed based on hierarchical conditions
- If none of the conditions is true then the final else block gets executed.
- Syntax:

```
if(condition1)
{
    //statements
}
else if(condition2)
{
    //statements
}
else
{
    //statemenst
}
```

4.) **Switch Statement:**

- It is used when multiple conditions have to be evaluated based on single variable expression.
- The switch statement evaluates an expression matching the switch value against the series of cases values and execute the statement after the first case matching value util a break statement is encountered.
- The default case of switch statement will be jump to if no case matches with the switch value.
- Syntax

```
Switch(expression)
{
        case 1:
                //statements
                break;
        case 2:
                //statements
                break;
        default:
                //statements
                break;
}
```

Example:

```javascript
const operator = String(prompt("Enter an operator"))
const a = Number(prompt("Enter a value"))
const b = Number(prompt("Enter b value"))

switch (operator) {
    case '+':
        console.log("addition of two number::", a + b)
        break;
    case '-':
        console.log("subraction of two number::", a - b)
        break;
    case '*':
        console.log("prodcut of two number::", a * b)
        break;
    case '/':
        console.log("division of two number::", a / b)
        break;
    case '%':
        console.log("modulous of two number::", a % b)
        break;
    default:
        console.log("Enter a valid operator")
        break;
}
```

- Note:
  1.) The switch statement accepts n number of cases
  2.) Cases are case-sensitive
  3.) Default value has the least priority.
  4.) The cases must be constant and unique.
  5.) The cases cannot be variable or expression.
  6.) The execution will flow through each case if break is missing in the satisfied case.

5.) **Looping Statement:**

- Whenever we want to perform a task repeatedly that time we can go with the looping statement.
- There are different types of looping statement in JavaScript.
  1.) for loop
  2.) while loop
  3.) do while loop
  4.) for-of
  5.) for-in

1) for loop:

- for loop it is used to execute set of statements repeatedly it is commonly used when we know how many times the loop needs to be executed.
- Syntax:

```
for (initialization; condition; updation)
{
    //set of statements
}
```

2) While loop:

- It loops through a block of code as long as specified condition is true.
- It commonly used when you don't know how many times you want to execute a block of code and it is based on the condition.
- Syntax:

```
Initialization
While(condition)
{
    //statements
    //updation

}
```

3) **Do while loop:**

- Do while loop will execute code of block once before checking the condition.
- If the condition is true then it will repeat the loop as long as the condition is true, and once condition is false it will stop the execution of the block
- It is commonly used when you have to execute the loop at least once.
- Syntax:

```
Initialization
do
{
//statements
//updation
} while(condition);
```

JAVASCRIPT

Ex:

```
let i=2;
do
{
   console.log(i)
   i+=2
} while(i<=100);
```

FUNCTIONS

- Reusable block of code that perform specific task.
- Define with function keyword, name, parameters (optional) and a body of code in curly braces.
- Called by using the function name and passing the argument (optional)
- Syntax:
- function funcitonName (list of arguments)

```
{
        //statements
}
functionName (list of arguments);
```

Example:

```
function sayHello(name)
{
   console.log (`Hello ${name}`)
}
sayHello("Dhanush");
```

Types of Functions:

1.) Named Function:

- This is the most common way of defining named function.
- Syntax:

```
function funcitonName (list of arguments)
{
        //statements
}
functionName (list of arguments);
```

Example:

```
function sayHello(name)
{
        console.log (`Hello ${name}`)
}
sayHello("JavaScript");
```

JAVASCRIPT

2.) Declaration Function:

- Declaration function is hoisted, which means they can be used before they defined in the code

- Example:

```
function mul (a, b)
{
        Console.log(a*b)
}
mul (10,20)
```

3.) Anonymous Function:

- A function declared without an identifier is known as anonymous function.
- To execute anonymous function, we have to store them into one variable.
- Syntax:

```
function ()
{
        //body
}
```

- Example:

```
let a = () => {
  console.log ("This is arrow function")
}
```

- To execute this function, we need to store the function in the variable and we need to invoke using that variable

4.) Function Expression

- Whenever we are storing any function into a variable then it is called as function expression.
- Syntax:

```
var a = function ()
{
        //body
}
a ();
```

- Example 1:

```
let a = () => {
  console.log ("This is arrow function")
}
a ();
```

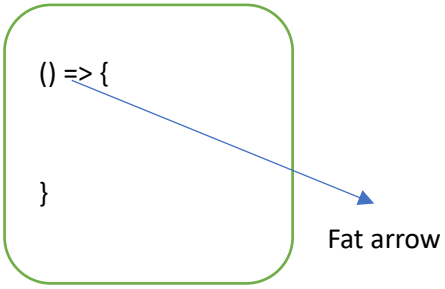
- Example 2:

```
let even=function(num)
{
  for (let i=2; i<=num; i+=2)
  {
    console.log(i)
  }
}
even(num);
```

JAVASCRIPT

5.) First class function:

- It is a function which is assigned as a value to a variable.
- It can be a named function or anonymous function or arrow function.
- It can be accessed only with the variable name, you cannot access it with the function name in case of named function
- Ex:
  Let a =



6.) Arrow Function:

- It is an advanced function.
- It will reduce the code
- To execute this function, we have to store it in a variable.
- Arrow functions are concise way to write functions in JavaScript which is introduced in ES6.
- They provide a cleaner syntax and can be especially useful for short syntax, anonymous function.
- Syntax 1:

```
(parameters) =>
{
        //code to be executed
}
```

- Key points of arrow functions
  1.) Concise syntax:
    (i) For single line functions with a single expression, you can avoid the curly braces and return keyword.
    (ii) Ex: const square = (x) => x*x;
    (iii) For a functions with multiple statements/ expressions or a block of code we need to use curly braces and return keyword if we have to return the value from that function.
  2.) Implicit return:
    (i) As mentioned, if you omit the curly braces the expression is implicitly returned.
    (ii) Ex: let add = (a, b) => a+b
    Console.log (add (10,20))
  3.) Parameter handling:
    (i) If you have a single parameter you can omit the parenthesis.
    (ii) Ex: let greet = name =>{
    Console.log(`goodafternoon ${name}`)
    } greet ();
    (iii) If you don't have any parameter then you can use underscore (_) or $.
    (iv) Ex: let greet = _ => Console.log (`good afternoon ${name}`)

JAVASCRIPT

(v) If we have the multiple parameters then you should use the parenthesis.
Ex: let sum= (a, b) => {return a+b }
Console.log (sum (a, b))

4.) This keyword:
(i) Arrow function inherit this value from their enclosing scope unlike regular functions.
(ii) This can be helpful in some scenarios especially when dealing with even handlers or asynchronous operations.

Advantages of arrow functions:

- Conciseness
- Readability
- This keyword binding.

When to use the arrow functions:

1.) Callback functions:
- They can be used as callback for functions like map, filters and reduce.

2.) Event Handlers:
- They can be used in event handlers to avoid this keyword binding issues

7.) Callback Function:

- It is a function which sends as a parameter to another function.
  - (or)
- callback functions are functions that are passed as a to arguments to another function
  - Syntax:

```
function callbackFunction () {
        //Set of statements
}
function mainfunciton (callback) {
        return Callback ();
}
```

  - Example:

```
function add (a, b)
{
  return a+b;
}
function Calculate (callback, a, b) {
  return callback (a, b)
}
let res=Calculate(add,10,20)
console.log(res)
```

  - You can send any function as the callback function.

8.) **Higher order function:**
- It is a function which will accept another function as an argument (or) it will return another function is called as Higher order function.
- Syntax:

```
function callbackFunction () {
        //Set of statements
}
function mainfunciton (callback) {
        return Callback ();
}
```

Higher Order function

- Example:

```
function add (a, b)
{
   return a+b;
}
function Calculate (callback, a, b) {
   return callback (a, b)
}
let res=Calculate(add,10,20)
console.log(res)
```

- Example 2:

```
function areaOfSquare(a)
{
   return a*a;
}
function mainFun (callback, a)
{
   return callback(a)
}
let a=10;
console.log ("The area of square is", mainFun (areaOfSquare, a))
```

- There are some inbuilt higher order functions present JavaScript map, reduce and filter, setTimeOut, setInterval, for each.

9.) Immediate Invoking function Expression (IIFE):

- IIFE function that is defined and executed immediately.
- Syntax:

```
(function () {
        //code inside IIFE
}) ();
```

- Example 1:

```
(function () {
   console.log ("this is immediate invoking functions")
}) (); //semi colon is mandatory if not given next IIFE function won't execute

(function () {
   console.log ("this is iikf 2")
})();
```

- Example 2:

```
(function () {
   let secretMessage="Chocolate is in the freezer"
   function showSecretMessage(secretMessage)
```

JAVASCRIPT

```
    {
      console.log(secretMessage)
    }
    showSecretMessage(secretMessage)
  })();
```

- It is used to create a private scope encapsulate variables and functions and avoid polluting the global name space.
- *How it works:*
  - The function () part defines an anonymous function.
  - Grouping Operator, the outer parenthesis groups the function expression.
  - Immediate invocation the final pair of parentheses immediately invokes the function

10.) **Nested Functions:**

- Nested functions are the functions defined within another function.
- Syntax:

```
function fun1() {
        //code
        function fun2() {
        //body of fun2
       }
    fun2()
}
fun1();
```

- Example:

```
 function Bank ()
 {
   let amount=3000;
   function deposit(deptamt)
   {
     let deposit=amount+deptamt
     return deposit;
   }
   return deposit(800)
 }
 console.log ("The total amt is", Bank ())
```

- They have access to the variable and the parameter of the outer function.

- **Lexical Scope:**
  - Nested function inherits the scope of the outer function this means they can access variables and parameters declared in the outer function.
  - **Example:**

```
 function createCounter()
 {
   let count=0;
   function increaseCount()
```


JAVASCRIPT

```
    {
        count++;
        return count
    }
    return increaseCount ();
}
let result=createCounter ()
console.log(result)
```

- **Scope chaining:**
  - In JavaScript scope chaining refers to the hierarchical structure of scopes that the JavaScript engine traverses to find the value of the variable or a function.
  - This chain starts from local scope and moves up to the global scope.
  - **Example:**

```
 let globalVariable="Global value"
function outerFunction () {
    let outerVariable="Outervalue"
    function innerFunction()
    {
        let innerVaribale="inner value"
        console.log(innerVaribale, outerVariable,globalVariable)
    }
    innerFunction()
}
outerFunction ()          //inner value outer value global value
```

11.)Function Currying:

- Function currying is a technique where a function takes one argument at a time and returns a new function that takes the next argument and so on this process continues until all arguments have been provided and the final function executes.
- Syntax**:**

```
 //? Syntax:
function fun1(parameter){
    return function fun2(parameter){
        return function fun3(parameter){
            return expression;
        }
    }
}
```

- Example Type-1:

```
 function sum () {
    return function(a) {
        return function(b) {
            return a+b
        }
    }
}
```

```
let res=sum () (10)(20)
console.log(res)
let result=sum ()
let data=result (10)
let data1=data (20)
console.log(data1)
```

- Example Type-2:

```
function calculateRateOfIntrest(p)
{
   return function(r) {
      return function(t) {
         return (p*t*r)/100
      }
   }
}
let SI=calculateRateOfIntrest (1000) (5)(2)
console.log (SI)
```

- Curried functions are mostly used to create higher order functions.
- They can help in writing more concise and readable code.
- Using currying function, you can break down complex functions into simpler, modular.

12.)**Generator Function:**

- Generator functions special type of functions in JavaScript that allow you to control the execution flow of a function, pausing and resuming it as needed.
- Pausing and resuming is achieved by using the **yield** keyword.
- Syntax:

```
function* genratorFunction()
{
   yield expression1;
   yield expression2;
   yield expression3;
}
let storeInsideVariable = genratorFunction()
console.log(storeInsideVariable.next().value)
```

- When a generator function is called it returns a generator object.
- Each time the next method is called on the generator object, the function executes until the next yield statement.
- When next method is called again the function resumes from the point where it was paused.
- **Example 1:**

```
function* functionGenerator ()
{
   yield 1
   yield 2
   yield 3
}
```

JAVASCRIPT

```
let x=functionGenerator ()
console.log(x.next().value)        //1
console.log(x.next().value)        //2
console.log(x.next().value)        //3
console.log(x.next().value)        //undefined
```

Arrays

1) An array in JavaScript is a data structure that stores a collection of elements.
2) These elements can be of various data types, including numbers, strings, objects, or even other arrays.
3) Arrays are ordered, meaning each element has a specific index associated with it, starting from 0.

Creating an Array:

There are two ways in creating an array in JavaScript.

1) Array Literal:
   let myArray = [1, 2, 3, "hello", true];
   This syntax directly initializes an array with the specified elements.
2) Using the Array constructor:
   let myArray = new Array(5); // Creates an array with 5 empty elements
   let myArray = new Array(1, 2, 3); // Creates an array with 3 elements

Accessing Array Elements:

You can access individual elements of an array using their index:

ex:

- let firstElement = myArray[0]; // Accesses the first element
- let lastElement = myArray [myArray.length - 1]; // Accesses the last element

Modifying Array Elements:

- You can modify elements of an array by assigning new values to their indices:
- myArray[2] = 10; // Changes the third element to 10

Array Methods in JavaScript:

Here are some of the most commonly used array methods

1.) push():

- Adds one or more elements to the end of an array.
- Syntax:
  - array.push(element1, element2, ...);
- ex: const numbers = [1, 2, 3];
- numbers.push(4, 5); // numbers becomes [1, 2, 3, 4, 5]
  - Return type: - new array length.

2.) pop():

- Removes the last element from an array and returns it.
- syntax:

```
array.pop();
```

- Ex: const numbers = [1, 2, 3];

```
const lastNumber = numbers.pop(); // lastNumber is 3, numbers becomes [1, 2]
```

- Return type: - the removed element.

3.) shift():

- Removes the first element from an array and returns it.
- syntax:

```
array.shift();
```

- ex:

```
const numbers = [1, 2, 3];
const firstNumber = numbers.shift(); // firstNumber is 1, numbers becomes
[2, 3]
```

- Return type: The removed element

4.) unshift();

- Adds one or more elements to the beginning of an array.
- syntax:

```
array.unshift(element1, element2, ...);
```

- Example:

```
const numbers = [2, 3];
numbers.unshift(1); // numbers becomes [1, 2, 3]
```

- Return type: The new length of the array.

5.) slice():

- Extracts a section of an array and returns a new array.
- syntax:
- array.slice(start, end);
- Example:

```
const numbers = [1, 2, 3, 4, 5];
const slicedArray = numbers.slice(1, 4); // slicedArray is [2, 3,4]
```

- Return type: A new array

6.) splice():

- Removes or replaces existing elements and/or adds new elements to an array.
- syntax:

```
array.splice(start, deleteCount, item1, item2, ...);
```

- Example:

```
const numbers = [1, 2, 3, 4, 5];
numbers.splice(2, 2, 6, 7); // numbers becomes [1, 2, 6, 7, 5]
```

- Return type: An array containing the removed elements

7.) contact()

- Merges two or more arrays and returns a new array.
- syntax:
- array.concat(array1, array2, ...);
- ex:

```
const array1 = [1, 2];
const array2 = [3, 4];
const mergedArray = array1.concat(array2); // mergedArray is [1,2, 3, 4]
```

- Return type: A new array.

JAVASCRIPT

8.) join():

- Joins all elements of an array into a string, separated by a specified separator.
- syntax:

```
array.join(separator);
```

- Example

```
const numbers = [1, 2, 3];
const string = numbers.join('-'); // string is "1-2-3"
```

- Return type: A string.

9.) reverse():

- Reverses the order of the elements in an array.
- syntax:
- array.reverse();
- ex:

```
const numbers = [1, 2, 3];
numbers.reverse(); // numbers becomes [3, 2, 1]
```

- Return type: The modified array.

10.)sort():

- Sorts the elements of an array.
- syntax:

```
array.sort(compareFunction);
```

- Example:

```
const numbers = [3, 1, 4, 1, 5, 9, 2, 6, 5, 3, 5];
numbers.sort(); // numbers becomes [1, 1, 2, 3, 3, 4, 5, 5, 5, 6,9]
```

- Return type: The modified array

11.)forEach():

- Executes a provided function once for each array element.
- syntax:
- array.forEach(callbackFunction);
- Example

```
const numbers = [1, 2, 3];
numbers.forEach(number => console.log(number));
```

- Return type: undefined.

12.)map():

- Creates a new array by transforming each element of the original array.
- Syntax:
- array.map(callbackFunction);
- Example:

```
const numbers = [1, 2, 3];
const doubledNumbers = numbers.map(number => number * 2); //
doubledNumbers is [2, 4, 6]
```

- Return type: A new array.

13.) filter():

- Creates a new array with elements that pass a test implemented by a provided function.
- Syntax:
- array.filter(callbackFunction);

JAVASCRIPT

- Example:
- const numbers = [1, 2, 3, 4, 5];
- const evenNumbers = numbers.filter(number => number % 2 === 0); // evenNumbers is [2, 4]
- Return type: A new array.

14.)reduce():

- Reduces an array to a single value.
- syntax:
- array.reduce(callbackFunction, initialValue);
- ex:

  const numbers = [1, 2, 3];

  const sum = numbers.reduce((accumulator, currentValue) => accumulator + currentValue, 0); // sum is 6
- Return type: A single value

15.)find():

- Returns the value of the first element in the array that satisfies the provided testing function.
- syntax:
- array.find(callbackFunction);
- Example:

  const numbers = [1, 2, 3, 4, 5];

  const firstEvenNumber = numbers.find(number => number % 2 === 0); // firstEvenNumber is 2
- Return type: The found element, or undefined if not found.

JavaScript Strings

In JavaScript we can create Strings in 4 ways:

1.) Using Double Quotes:
- Strings can be created using double quotes, which are one of the most common ways to represent strings in JavaScript.
- This method is suitable for strings that do not include embedded double quotes. If double quotes are required within the string, they need to be escaped using a backslash (\).
- Advantages: Simple and widely used.
- Example:
- let str1 = "Shourya someone";
- console.log(str1); // Output: Shourya someone

2.) Using Single Quotes:
- Strings can also be created using single quotes.
- They are especially useful when the string itself contains double quotes, as single quotes eliminate the need for escaping the double quotes.
- If the string contains single quotes, they need to be escaped.
- Advantages: Useful for embedding double quotes directly.

JAVASCRIPT

- Example:
- let str2 = ' "Shourya someone" ';
- console.log(str2); // Output: "Shourya someone"
- let str3 = 'Shourya\'s profile';
- console.log(str3); // Output: Shourya's profile

3.) Using Backticks (Template Literals)

- Backticks, also known as template literals, allow more dynamic string creation.
- They enable variable interpolation using ${} and support multi-line strings without the need for concatenation or escape characters.
- This makes them highly versatile for modern JavaScript.
- Advantages:
  1.) Cleaner syntax for including variables or expressions.
  2.) Supports multi-line strings natively.
- Example:

```
let myname = 'Shourya 2.0';
let str3 = `My name is ${myname}`;
console.log(str3); // Output: My name is Shourya 2.0
```

4.) JavaScript String() constructor:

- The JavaScript String() Constructor is used to can be used as a constructor or a function. that creates a new string object. It can be used in two different ways:
- Syntax:
- Invoked with the new keyword:
- new String(object);
- Ex:

```
// Using with new keyword
let str = new String("Hello");
console.log(str);
```

- Invoked without the new keyword:
- String(object);
- Ex: // Using without keyword

```
let strValue = String("World");
console.log(strValue);
```

Parameters:
-object: This parameter contains a value that is to be converted to a string value.
Return Value:
-When the String() constructor is used with the new keyword to create a new string object, it returns the newly created string object.
-when the String() constructor is used without the new keyword, it behaves differently and returns a primitive string value rather than a string object.

JavaScript String Methods: Notes

1.) charAt():

- Syntax: string.charAt(index)
- Description: Returns the character at the specified index.
- Return Type: String
- Example:

```
let myname = 'Shourya 2.0';
console.log(myname.charAt(0)); // S
console.log(myname.charAt(1)); // h
```

2.) charCodeAt()
- Syntax: string.charCodeAt(index)
- Description: Returns the Unicode value of the character at the specified index.
- Return Type: Number
- Example:

```
let myname = 'Shourya 2.0';
console.log(myname.charCodeAt(0)); // 83
```

3.) concat()
- Syntax: string.concat(string2, string3, ...)
- Description: Concatenates two or more strings.
- Return Type: String
- Example:

```
let myname = 'Shourya';
let surname = ' Shetty';
console.log(myname.concat(surname)); // Shourya Shetty
```

4.) includes()
- Syntax: string.includes(substring)
- Description: Checks if a substring is present in the string.
- Return Type: Boolean
- Example:

```
let myname = 'Shourya Shetty';
console.log(myname.includes("Shetty")); // true
console.log(myname.includes("xyz"));   // false
```

5.) trim()
- Syntax: string.trim()
- Description: Removes whitespace from both ends of a string.
- Return Type: String
- Example:

```
let myname = '   Shourya Shetty   ';
console.log(myname.trim()); // "Shourya Shetty"
```

6.) trimEnd()
- Syntax: string.trimEnd()
- Description: Removes whitespace from the end of a string.
- Return Type: String
- Example:

```
let myname = '   Shourya Shetty   ';
console.log(myname.trimEnd()); // "   Shourya Shetty"
```

7.) trimStart()
- Syntax: string.trimStart()
- Description: Removes whitespace from the start of a string.
- Return Type: String
- Example:

```
let myname = '   Shourya Shetty   ';
console.log(myname.trimStart()); // "Shourya Shetty   "
```

8.) repeat()

JAVASCRIPT

- Syntax: string.repeat(count)
- Description: Repeats the string the specified number of times.
- Return Type: String
- Example:

```
let myname = "Shourya ";
console.log(myname.repeat(3)); // "Shourya Shourya Shourya "
```

9.) replace()

- Syntax: string.replace(oldSubstring, newSubstring)
- Description: Replaces a specified substring with a new one.
- Return Type: String
- Example:

```
let website = 'Visit MicroSoft';
console.log(website.replace("MicroSoft", "Google")); // Visit Google
```

10.)indexOf()

- Syntax: string.indexOf(substring)
- Description: Returns the index of the first occurrence of a specified substring.
- Return Type: Number
- Example:

```
let website = 'Visit MicroSoft';
console.log(website.indexOf("MicroSoft")); // 6
```

11.)substring()

- Syntax: string.substring(startIndex, endIndex)
- Description: Extracts a portion of the string between specified indexes.
- Return Type: String
- Example:

```
let website = 'Visit MicroSoft';
console.log(website.substring(2, 7)); // "sit M"
```

12.)substr()

- Syntax: string.substr(startIndex, length)
- Description: Extracts a portion of the string starting at a specified index and spanning a specified number of characters.
- Return Type: String
- Example:

```
let website = 'Visit MicroSoft';
console.log(website.substr(6, 2)); // "Mi"
```

13.)toUpperCase()

- Syntax: string.toUpperCase()
- Description: Converts the string to uppercase.
- Return Type: String
- Example:

```
let website = 'Visit MicroSoft';
console.log(website.toUpperCase()); // VISIT MICROSOFT
```

14.)toLoweCase()

- Syntax: string.toLowerCase()
- Description: Converts the string to lowercase.
- Return Type: String
- Example:

```
let website = 'Visit MicroSoft';
```

JAVASCRIPT

console.log(website.toLowerCase()); // visit Microsoft

15.)startsWith()

- Syntax: string.startsWith(substring)
- Description: Checks if the string starts with the specified substring.
- Return Type: Boolean
- Example:

```
let website = 'Visit MicroSoft';
console.log(website.startsWith("Visit")); // true
```

16.)endsWith()

- Syntax: string.endsWith(substring)
- Description: Checks if the string ends with the specified substring.
- Return Type: Boolean
- Example:

```
let website = 'Visit MicroSoft';
console.log(website.endsWith("Soft")); // true
```

17.)slice()

- Syntax: string.slice(startIndex, endIndex)
- Description: Extracts a portion of the string based on specified indexes.
- Return Type: String
- Example:

```
let website = 'Visit MicroSoft';
console.log(website.slice(6, 11)); // "Micro"
```

18.)split():

- Syntax: string.split(delimiter)
- Description: Splits the string into an array based on the specified delimiter.
- Return Type: Array
- Example:

```
let message = "How are you Shourya and someone";
console.log(message.split(" ")); // ["How", "are", "you", "Shourya", "and",
"someone"]
```

19.)search()

- Syntax: string.search(pattern)
- Description: Searches for a pattern and returns the index of its first occurrence.
- Return Type: Number
- Example:

```
let message = "How are YOU Shourya and someone";
console.log(message.search(/you/i)); // 8
```

JAVASCRIPT

JavaScript Object

Object:

- An object in JavaScript is a collection of properties, where each property is a key-value pair.
- Objects are one of the fundamental building blocks in JavaScript, allowing you to group related data and functions together.

Rules for defining key in Object:

- Keys can be written without quotes if they are valid JavaScript identifiers (e.g., name, age).
- Use quotes (single or double) for keys with spaces or special characters (e.g., "first name", "age#").
- Keys are case-sensitive (e.g., name and Name are different).

Rules for defining values in Object:

- Values can be strings, numbers, Booleans, objects, arrays, functions, etc.
- Any valid datatype in js we can store as value.

Accessing the properties of an object:

- There are 2 ways to access the object properties:
  1.) Dot Notation.
  2.) Bracket Notation.

1.) Dot Notation:

- The property name must be a valid JavaScript identifier (letters, numbers, underscores, or $).
- Cannot be used if the property name has spaces or special characters.
- Syntax:

```
object.propetry;
```

- Example:

```
const employee = {
 name: "Raj",
 age: 30,
 department: "HR"
};
console.log(employee.name); // Raj
console.log(employee.age);  // 30
```

2.) Bracket Notation:

- The property name is provided as a string inside brackets.
- Useful for accessing properties with names that include spaces or special characters.
- Allows dynamic access using variables.
- Syntax:

```
object["property"]
```

- Example:

```
const employee = {
 "full name": "Raj Bhosale",
```

```
    age: 30,
    department: "HR"
};console.log(employee["full name"]); // Raj Bhosale
```

Delete Operator:

- The delete operator removes a property from an object.
- Syntax:
- delete object.property;
- Example:

```
const employee = { name: "Raj", age: 30 };

delete employee.age;

console.log(employee); // {name: "Raj"}
```

Ways of Creating Objects in JavaScript:

1.) Literal way of creating object.
   - This is the most common and straightforward way to create objects in JavaScript.
   - You can define an object using curly braces {} and specify its properties directly.
   - Object literal syntax is concise and intuitive. It allows you to create an object with key-value pairs.
   - The key is a string (or symbol) and the value can be any valid JavaScript type (number, string, function, array, etc.).
   - Syntax:

```
Variable_decleration objectName = {
    key: value,
    key2: value2
};
```

   - Example:

```
const marker = {
    color: "black",
    price: 20,
    brand: "doms",
    height: "10cm"
};
console.log(marker);
```

2.) Using new Object:
   - You can create an empty object by using the Object constructor, which is a built-in function.
   - new Object() creates an empty object and then you can manually assign properties to it.
   - Syntax:

```
Variable_Decleration variable_name=new Object()
```

   - Example:

```
let user = new Object();
user.name = "Shourya";
user.sayHello = function() {
    console.log("Hello!");
```

JAVASCRIPT

```
};
console.log(user);
user.sayHello();
```

3.) Using Constructor Functions:

- Constructor functions allow you to create multiple instances of similar objects.
- These functions are used to initialize an object with specific properties and methods.
- The new keyword creates a new empty object.
- This keyword refers to the newly created object.
- Properties and methods are assigned to the new object.
- The newly created object is returned.
- Syntax:

```
function ConstructorName(parameter1, parameter2, ...) {
    this.property1 = parameter1;
    this.property2 = parameter2;
    // other properties and methods
     this.method = function() {
      // method code
     };
  }
Variable_decleration instanceName = new ConstructorName(parameter1,
parameter2, ...);
```

- Example:

```
function Employee(id, name, age, salary) {
this.id = id;
this.name = name;
this.age = age;
this.salary = salary;
}

let emp1 = new Employee(101, "Raj", 30, 5000);
let emp2 = new Employee(102, "Vishal", 28, 5100);
console.log(emp1);
console.log(emp2);
```

Methods in JavaScript:

- In JavaScript, a method is a function that is associated with an object.
- When a function is defined inside an object, it's called a method of that object.
- The syntax for a method is the same as that of a function, but it's usually referenced through the object..
- Methods are defined within an object. You can define a method in an object literal using the function.
- Example:

```
let user = {
name: "Shourya",
sayHello: function() {
console.log("Hello!");
}
};
```


JAVASCRIPT

```
user.sayHello();
```

-ES6 shorthand syntax allows a cleaner way to define methods:

```
const person = {
  name: 'John',
  greet() {
    console.log('Hello, ' + this.name);
  }
};

person.greet(); // Output: Hello, John
```

JavaScript Object Methods:

1.) keys():
   - Syntax:
     ```
     Object.keys(obj)
     ```
   - Description:
     Returns an array of the object's own enumerable property names.
   - Return Type: Array
   - Example:
     ```
     let user = {
        name: "Shourya",
        age: 23,
        city: "Cyberabad"
     };
     let keys = Object.keys(user);
     console.log(keys); // ["name", "age", "city"]
     for (let i = 0; i < keys.length; i++) {
        console.log(user[keys[i]]); // Accessing values using keys
     }
     ```
2.) values():
   - Syntax:
     ```
     Object.values(obj);
     ```
   - Description:
     Returns an array of the object's own enumerable property values.
   - Return type: Array
   - Example:
     ```
     let user = {
        name: "Shourya",
        age: 23,
        city: "Cyberabad"
     };
     let values = Object.values(user);
     console.log(values); // ["Shourya", 23, "Cyberabad"]
     ```
3.) Entries():

   - Syntax: Object.entries(obj)

JAVASCRIPT

- Description: Returns an array of the object's own enumerable property [key, value] pairs.
- Return Type: Array of arrays.
- Examples:

```
let user = {
   name: "Shourya",
   age: 23,
   city: "Cyberabad"
};
let entries = Object.entries(user);
console.log(entries); // [["name", "Shourya"], ["age", 23], ["city", "Cyberabad"]]
```

4.) hasOwn():

- Syntax: Object.hasOwn(obj, prop)
- Description: Checks whether the specified property exists in the object as its own (not inherited) property.
- Return Type: Boolean
- Example:

```
let user = {
   name: "Shourya",
   age: 23
};
let isPresent = Object.hasOwn(user, "name");
console.log(isPresent); // true
```

5.) seal():

- Syntax: Object.seal(obj)
- Description: Prevents adding or deleting properties but allows modifying existing properties of the object.
- Return Type: Object.
- Example:

```
let user = {
   name: "Shourya",
   age: 23
};
Object.seal(user);
user.name = "Rahman"; // Allowed
delete user.age; // Not allowed
user.isMarried = false; // Not allowed
console.log(user); // {name: "Rahman", age: 23}
```

6.) isSealed():

- Syntax: Object.isSealed(obj)
- Description: Checks if the object is sealed.
- Return Type: Boolean
- Example:

```
let user = {
   name: "Shourya",
   age: 23
};
Object.seal(user);
```

JAVASCRIPT

```
console.log(Object.isSealed(user)); // true
```

7.) freeze()

- Syntax: Object.freeze(obj)
- Description: Prevents adding, deleting, or modifying properties of the object.
- Return Type: Object
- Example:

```
let user = {
  name: "Shourya",
  age: 23
};
Object.freeze(user);
user.name = "Rahman"; // Not allowed
delete user.age; // Not allowed
user.isMarried = false; // Not allowed
console.log(user); // {name: "Shourya", age: 23}
```

8.) isFrozen():

- Syntax: Object.isFrozen(obj)
- Description: Checks if the object is frozen.
- Return Type: Boolean
- Example:

```
let user = {
  name: "Shourya",
  age: 23
};
Object.freeze(user);
console.log(Object.isFrozen(user)); // true
```

9.) assign():

- Syntax: Object.assign(target, ...sources)
- Description: Used to copy properties from one or more source objects to a target object. Primarily useful for merging objects.
- Original objects remain unchanged
- Return Type: Returns the target object after merging.
- Example:

```
const employeeDetails = {id: 101, name: "Alice"};
const employeeJob = {position: "Developer", department: "IT"};
const employeeContact = {email: "alice@example.com", phone: "123-456-7890"};
const completeEmployee = Object.assign({}, employeeDetails, employeeJob, employeeContact);
console.log(completeEmployee);
```

Output:

```
{
  id: 101,
  name: "Raj",
  position: "Developer",
  department: "IT",
  email: "Raj@example.com",
  phone: "123-456-7890"
```

JAVASCRIPT

```
}
console.log(employeeDetails); // { id: 101, name: "Raj" }
console.log(employeeJob);     // { position: "Developer", department: "IT" }
console.log(employeeContact); // { email: "Raj@example.com", phone:
"123-456-7890" }
```

10.)Object.create():

- Object.create() is a method used to create a new object.
- It allows you to set an existing object as the prototype of the new object. This is useful for inheritance.
- Syntax: Object.create(object, [propertiesObject])
- object: The object to use as the prototype for the new object (or null for no prototype).
- propertiesObject: (Optional) An object defining new properties to add to the new object.
- Example:

```
const animal = {
  eat: function() {
    console.log("Eating...");
  }
};

// Create a new object that inherits from `animal`
const dog = Object.create(animal);
dog.bark = function() {
  console.log("Barking...");
};

dog.eat(); // Eating...
dog.bark(); // Barking...
```

Shallow Copy in JavaScript:

- A shallow copy in JavaScript refers to a method of copying an object or an array where the top-level properties are duplicated, but any nested objects or arrays are still referenced from the original. This means that if you modify a nested object in the shallow copy, it will also affect the original
- Object because both share the same reference to that nested object.
- Example:

```
// Original object with a nested object
let originalObject = {
        name: "apple",
        price: {
              chennai: 120
            }
  };

// Creating a shallow copy using the spread operator
let clonedObject = { ...originalObject };
```

```
// Modifying the nested object in the cloned copy
clonedObject.price.chennai = 100;

// Output the values to see the effect
console.log("Cloned Object:", clonedObject); // Output: { name: "apple",
price: { chennai: 100 } }
console.log("Original Object:", originalObject); // Output: { name: "apple",
price: { chennai: 100 } }
```

Deep Copy in JavaScript:

- A deep copy in JavaScript refers to the process of creating a completely independent copy of an object or array, including all nested objects and arrays.
- This means that changes made to the deep copy do not affect the original object, as all levels of the structure are duplicated.

1. deep copy using JSON.stringify() and JSON.parse().
   - The most common way to create a deep copy is by using JSON.stringify() and JSON.parse().
   - This method converts an object into a JSON string and then parses it back into a new object.
   - const original = {

```
    name: 'Raj',
    age: 25,
    address: {
      city: 'Banglore',
      country: 'India'
    }
  };
  const deepCopy = JSON.parse(JSON.stringify(original));
  deepCopy.address.city = 'Hyderabad';
  console.log(original.address.city); // Output: Banglore
```

2. structuredClone():
   - The structuredClone() function is a modern way to create deep copies.
   - It handles more complex types and is built into JavaScript.
   - Example:

```
const original = { a: 1, b: { c: 2 } };
const cloned = structuredClone(original);
cloned.b.c = 3;
console.log(original.b.c); // Output: 2
```

JAVASCRIPT

This keyword:

1. Global Context:

```
// Non-strict mode
console.log(this); // window (in browsers)

// Strict mode
"use strict";
console.log(this); // undefined
```

2. Inside a Function:
   - Non-Strict Mode: Refers to the global object.
   - Strict Mode: this is undefined

```
// Non-strict mode
function myFunction() {
console.log(this); // window
}
myFunction();

// Strict mode
"use strict";
function myStrictFunction() {
console.log(this); // undefined
}
myStrictFunction();
```

3. Inside a Method (Object Context):
   - In arrow functions, the `this` keyword is lexically bound, meaning it inherits `this` from the surrounding (non-arrow) function or the global context where the arrow function is defined
   - Example:

```
function outerFunction() {
const arrowFunc = () => console.log(this); // Inherits `this` from the enclosing function
arrowFunc();
}
outerFunction();
const obj = {
name: "Raj",
getName: () => {
console.log(this.name); // `this` here depends on the outer context (likely `undefined` or `window`).
},
};
obj.getName()
```

this keyword in method:

- This keyword inside a method refers to the object that the method is a part of

JAVASCRIPT

- This allows you to access the object's properties and other methods from within.
- Example:

```
const car = {
    make: 'Toyota',
        model: 'Corolla',
    describe() {
      console.log(`This car is a ${this.make} ${this.model}.`);
    }
   };
car.describe(); // Output: This car is a Toyota Corolla.
```

JSON Object:

1.) JSON means JAVASCRIPT OBJECT NOTATION
2.) JSON is a light weight, text-based data interchange format that is easy for humans to read and write and easy for machines to parse and generate
3.) It is derived from syntax of JS object literals.

uses of JSON:

1.) Data transmission
    I. Server site to client site:
        - JSON is commonly used to send data from server to web client
        - Server send data into the JAson format which is then transmitted over the network
        - The client site JS code parses the JASAon format into JS object to access and manipulate the data
    II. Client site to server site:
        - JSON can also be used to send data from client to server, such as form data or user data
        - JS code on client site converts the data into the JSON format and send it to the server in a request.
2.) Data Storage:
    - JSON can be used to store the data in files with filename.json

JAVASCRIPT

Asynchronous JavaScript

Asynchronous JavaScript is a programming technique that allows JavaScript programs to run multiple tasks simultaneously, rather than executing them one after the other.

OR

Asynchronous JavaScript allows the execution of tasks without blocking the main thread. This non-blocking nature enables JavaScript to perform other tasks while waiting for long-running operations to complete, improving efficiency and user experience.

Key Characteristics of Asynchronous JavaScript:

1.) Non-Blocking:
   - Asynchronous code lets other tasks run while waiting for operations like network requests to complete.

2.) Concurrency:
   - JavaScript uses the event loop to manage tasks, making it feel like multiple tasks run at the same time, even though it's single-threaded.

3.) Improved Performance:
   - Multiple tasks can be handled without slowing down or freezing the user interface.

1.) Time function:(web api)

   i. setTimeOut():

   - setTimeOut is a JavaScript function that schedules a single execution of a function after a specified delay (in milliseconds). The function will be executed once after the delay is over.
   - Syntax:

```
setTimeout(callback, delay, arg1, arg2, ...);
```

1. callback: The function to execute.
2. delay: Delay in milliseconds (default is 0).
3. arg1, arg2, ...: (Optional) Arguments to pass to the callback.

   - These parameters will be passed in the order in which they are given when the function is invoked.
   - Example:

```
setTimeout(() => {
        console.log("Executed after 2 seconds");
    }, 2000);
```

   - The return type of setTimeout() is a numeric identifier called a Timeout ID.
   - This ID is used with clearTimeout() to cancel the timer.
   - Example 2:

```
const timeoutId = setTimeout(() => {
        console.log("This will run after 2 seconds");
    }, 2000);

console.log(typeof timeoutId); // "number"
```

   ii. clearTimeout():

- The clearTimeout() function in JavaScript clears the timeout which has been set by the setTimeout()function before that.
- Syntax:

```
clearTimeout(timeoutId);
```

- timeoutId: it is return by the setTimeOut().
- Example:

```
<button onclick="start()" >Start </button>
 <button onclick="stop()" >Stop </button>

<script>
        let timeoutId;
        function start() {
          timeoutId=setTimeout(()=>{
         document.write("<h1>Hello set timeout</h1>")
         },3000)
        }

        function stop() {
        clearTimeout(timeoutId);
        alert("time out stopped")
       }
</script>
```

- Real world Use Cases:
  1. Cancel a Task on User Action:
     Example: Cancelling a notification or warning when the user responds quickly.
  2. Prevent Duplicate Actions:
     Example: Clearing a timeout to debounce user inputs or clicks.

iii. setInterval():

- The setInterval function in JavaScript is commonly used for repeating a task at fixed intervals.
- Syntax:

```
setInterval(callback, delay, arg1, arg2, ...);
```

  1. callback: The function to be executed repeatedly.
  2. delay: The time interval (in milliseconds) between each execution.
  3. arg1, arg2, ...: (Optional) Arguments to pass to the callback function.
- it will return a numeric interval ID, which can be used with clearInterval() to stop the repeated execution.
- Example:

```
setInterval(() => {
        const now = new Date();
        console.log(now.toLocaleTimeString());
}, 1000);
```

iv. clearInterval():

- The clearTimeout() function in javascript clears the timeout which has been set by the setTimeout()function before that.
- Syntax:

```
clearInterval(intervalId);
```

- Example:

```
let counter = 0;
const intervalID = setInterval(() => {
console.log(counter);
counter++;
    if (counter === 5) {
        clearInterval(intervalID);
        console.log("Interval cleared.");
    }
}, 1000);
```

AJAX (Asynchronous JavaScript and XMLHttpRequest):

- AJAX is a technique used in web development to send and receive data asynchronously between the client (browser) and the server without reloading the entire page. This allows for more dynamic and interactive web applications.
- The traditional way to make AJAX requests.
- It's a built-in JavaScript object that allows sending and receiving data asynchronously.
- It is used to send and receive the data from backend.
- There are few steps we have to follow when working with AJAX.

1.) Create an XMLHttpRequest object using XMLHttpRequest() constructor.

```
let xhr=new XMLHttpRequest();
```

2.) Create a request :
   - Use the open() method to specify the HTTP request type (GET, POST, etc.), the URL, and whether the request should be asynchronous.
   - For get method:

```
xhr.open('GET', 'apiUrl', true);
```

   - for post method:

```
xhr.open('POST', 'apiurl', true);
```

3.) Only for Post Request:
   - xhr.setRequestHeader('Content-Type', 'application/json'); // Set content type for JSON
4.) Send the Request:
   - Use the send() method to send the request to the server. If it's a GET request, you don't need to send any data. For POST requests, you can pass the data as an argument.
   - if it is Get method

```
xhr.send();
```

   - if it is Post method

```
xhr.send(JSON.stringify(data));
```

5.) Set up the callback function to handle the response or error:
- for example:

```
xhr.onload = function () {
if (xhr.status === 200) {
 }
};

xhr.onerror=function () {
   console.log("Request failed")
}
```

Promise

- The Promise is an object represents the eventual completion (or failure) of an asynchronous operation and its resulting value.
- A promise object has a state that can be one of the following:
  1.) Pending
  2.) Fulfilled with a value
  3.) Rejected for a reason
- In the beginning, the state of a promise is pending, indicating that the asynchronous operation is in progress.
- Depending on the result of the asynchronous operation, the state changes to either fulfilled or rejected.
- The fulfilled state indicates that the asynchronous operation was completed successfully:
- The rejected state indicates that the asynchronous operation failed.

Steps for Working with Promises in JavaScript:

1.) Creating a Promise:
- To create a promise object, you use the Promise() constructor:
- Syntax:

```
const promise = new Promise((resolve, reject) => {

    // contain an operation

  if (success) {
      resolve(value);
  }
  else {
      reject(error);
  }
});
```

- The promise constructor accepts a callback function that typically performs an asynchronous operation. This function is called as an executor function.
- The executor function accepts two callback functions with the name resolve and reject.

JAVASCRIPT

- If the asynchronous operation completes successfully, the executor will call the resolve() function to change the state of the promise from pending to fulfilled with a value.
- In case of an error, the executor will call the reject() function to change the state of the promise from pending to rejected with the error reason.
- Once a promise reaches either a fulfilled or rejected state, it stays in that state and can't go to another state.
- A promise cannot go from the fulfilled state to the rejected state and vice versa.
- Also, it cannot go back from the fulfilled or rejected state to the pending state.

```
                                |---------resolve()--------> fulfilled
                                |
promise ====  pending =======>
                                |
                                |---------reject()----------> rejected
```

2.) Consuming a Promise:

- Consuming a Promise means handling the result of a Promise once it's done. A Promise can either succeed or fail, and you need to tell JavaScript what to do when that happens.
- **1) Consuming a Promise with then() method:**
- To get the value of a promise when it's fulfilled, you call the then() method of the promise object.
- Syntax:

```
promise.then(onFulfilled,onRejected);
```

- The then() method accepts two callback functions: onFulfilled and onRejected.
- The then() method calls the onFulfilled() with a value, if the promise is fulfilled.
- The then() method calls the onRejected() with an error if the promise is rejected.
- Note: both onFullfilled and onRejected are optional.
- Example:

```
function getUsers() {
        return new Promise((resolve, reject) => {
            setTimeout(() => {
                resolve([
                    { username: 'Raj', email: 'Raj@gmail.com' },
                    { username: 'Aajay', email: 'Aajay@gmail.com' },
                ]);
            }, 1000);
        });
    }

function onFulfilled(users) {
        console.log(users);
    }

function onRejected(error) {
        console.log(error);
```

```
}


const promise = getUsers();
promise.then(onFulfilled,onRejected);
```

- 2. **Consuming a Promise with catch() method:**
- If you want to get the error only when the state of the promise is rejected, you can use the catch() method of the Promise object.
- Syntax of catch() method:

```
promise.catch(onRejected);
```

- Example:

```
let success = false;

function getUsers() {
    return new Promise((resolve, reject) => {
        setTimeout(() => {
            if (success) {
                resolve([
                    { username: 'john', email: 'john@test.com'
},
                    { username: 'jane', email: 'jane@test.com'
},
                ]);
            }
            else {
                reject('Failed to the user list');
            }
        }, 1000);
    });
}

const promise = getUsers();

promise.catch((error) => {
    console.log(error);
});
```

3.) The finally() method:

- Sometimes, you want to execute the same piece of code whether the promise is fulfilled or rejected.
- Syntax:

```
promise.finally(callbackfun)
const render = () => {
    //...

};

getUsers()

.then((users) => {
```


JAVASCRIPT

```
                console.log(users);
        })
        .catch((error) => {
                console.log(error);
        })
        .finally(() => {
                render();
        });
```

Promise Chaining:

- Sometimes, you want to execute two or more related asynchronous operations, where the next operation starts with the result from the previous one.
- Promise chaining is a programming pattern in JavaScript used to handle sequences of asynchronous operations where each subsequent operation starts only after the previous one completes.
- This is done by chaining .then() handlers to a promise.
- Each .then() returns a new promise, allowing subsequent .then() calls to form a chain.
- Syntax:
  step1().then(result => step2(result)).then(result => step3(result))...
- Example:

```
let p = new Promise((resolve, reject) => {
        setTimeout(() => {
            resolve(10);
        }, 3 * 100);
    });

  p.then((result) => {
        console.log(result);
        return result * 2;
    });
```

- The callback passed to the then() method executes once the promise is resolved. In the callback, we show the result of the promise and return a new value multiplied by two (result*2).
- Because the then() method returns a new Promise with a value resolved to a value, you can call the then() method on the return Promise like this.

```
let p = new Promise((resolve, reject) => {
        setTimeout(() => {
            resolve(10);
        }, 3 * 100);
    });

  p.then((result) => {
        console.log(result);
        return result * 2;
        }).then((result) => {
      console.log(result);
```

```
    return result * 3;
});
```

- In this example, the return value in the first then() method is passed to the second then() method. You can keep calling the then() method successively.
- The way we call the then() method like this is often referred to as a promise chain.

Returning a Promise:

- When you return a value in the then() method, the then() method returns a new Promise that immediately resolves to the return value.

Promise Methods:

1.) Promise.all():

- The Promise.all() method returns a single promise that resolves when all the input promises have been resolved.
- The Promise.all() static method takes an iterable of promises:
- Syntax: Promise.all(iterable);
- In other words, the Promise.all() waits for all the input promises to resolve and returns a new promise that resolves to an array containing the results of the input promises.
- If one of the input promises is rejected, the Promise.all() method immediately returns a promise that is rejected with an error of the first rejected promise:
- Example

1) **Resolved promises example:**

```
const p1 = new Promise((resolve, reject) => {
setTimeout(() => {
  console.log('The first promise has resolved');
  resolve(10);
}, 1 * 1000);
});
const p2 = new Promise((resolve, reject) => {
setTimeout(() => {
  console.log('The second promise has resolved');
  resolve(20);
}, 2 * 1000);
});
const p3 = new Promise((resolve, reject) => {
setTimeout(() => {
  console.log('The third promise has resolved');
  resolve(30);
}, 3 * 1000);
});

Promise.all([p1, p2, p3]).then((results) => {
const total = results.reduce((p, c) => p + c);

console.log(`Results: ${results}`);
```

JAVASCRIPT

```
console.log(`Total: ${total}`);
});
```

2**) Rejected promises example:**

```
const p1 = new Promise((resolve, reject) => {
  setTimeout(() => {
     console.log('The first promise has resolved');
     resolve(10);
  }, 1 * 1000);

});
const p2 = new Promise((resolve, reject) => {
  setTimeout(() => {
     console.log('The second promise has rejected');
     reject('Failed');
  }, 2 * 1000);
});
const p3 = new Promise((resolve, reject) => {
  setTimeout(() => {
     console.log('The third promise has resolved');
     resolve(30);
  }, 3 * 1000);
});


Promise.all([p1, p2, p3])
  .then(console.log) // never execute
  .catch(console.log);
   Promise.race([p1, p2])
  .then(value => console.log(`Resolved: ${value}`))
  .catch(reason => console.log(`Rejected: ${reason}`));
```

```
o/p: The first promise has resolved
     Resolved: 10
     The second promise has resolved
```

2.) Promise.any():

- If one of the promises in the iterable object is fulfilled, the Promise.any() returns a single promise that resolves to a value which is the result of the fulfilled promise.
- The Promise.any() method accepts a list of Promise objects as an iterable object.
- syntax:

```
Promise.any(iterable);
```

- The Promise.any() returns a promise that is fulfilled with any first fulfilled promise even if some promises in the iterable object are rejected:
- Example:

```
const p1 = new Promise((resolve, reject) => {
    setTimeout(() => {
    console.log('Promise 1 fulfilled');
```


JAVASCRIPT

```
    resolve(1);
  }, 1000);
 });

const p2 = new Promise((resolve, reject) => {
 setTimeout(() => {
   console.log('Promise 2 fulfilled');
   resolve(2);
 }, 2000);
 });

 const p = Promise.any([p1, p2]);
 p.then((value) => {
 console.log('Returned Promise');
 console.log(value);
 });
```

Async and await:

- ES2017 introduced the async/await keywords that build on top of promises, allowing you to write asynchronous code that looks more like synchronous code and is more readable.
- Technically speaking, the async / await is syntactic sugar for promises.

async Keyword:

- The async keyword allows you to define a function that handles asynchronous operations.
- To define an async function, you place the async keyword in front of the function keyword as follows:
- Asynchronous functions execute asynchronously via the event loop. It always returns a Promise.
- Example:

```
async function sayHi() {
        return 'Hi';
    }
```

- In this example, because the sayHi() function returns a Promise, you can consume it, like this:

```
sayHi().then(console.log);
```

- You can also explicitly return a Promise from the sayHi() function as shown in the following code:

```
 async function sayHi() {
    return Promise.resolve('Hi');
  }
```

await keyword:

- You use the await keyword to wait for a Promise to settle either in a resolved or rejected state.
- You can use the await keyword only inside an async function.

- Example:

```
async function display() {
  let result = await sayHi();
  console.log(result);
 }
```

async function:

- An async function in JavaScript is a function that allows you to work with asynchronous code in a more readable and structured way.
- It enables the use of the await keyword, which pauses the execution of the function until a Promise is resolved or rejected.
- Use the async keyword before the function keyword to define an async function.
- Example:

```
async function example() {
        return "Hello, Async!";
    }
```

- An async function always returns a Promise. If the function explicitly returns a value, the Promise is resolved with that value.

BOM:(Browser Object Model)

- The Browser Object Model (BOM) is the core of JavaScript on the web.

- Represents the browser window and serves as the global object in JavaScript.

- The Browser Object Model (BOM) in JavaScript is a collection of objects that allows developers to interact with the web browser.

- It provides functionalities to manipulate the browser window, history, navigation, and other aspects of the browser environment.

- Window object is present inside Browser.

- Browser is platform for client-side application.

- whenever we are running the application then browser creates Web Api (Provided by browser).

```
                                        Browser
                                          ||
                                          ||
                                          ||
                                     Window Object
---------------------------------------------- ||-------------------------------------------------------------------
|       |       |         |              |              |            |               |           |

alert   Prompt  confirm   screen Object   Location Object   Navigator Object   History Object   Window size  document
```

Window Object:

- The global object of JavaScript in the web browser is the window object.
- It means that all variables and functions declared globally with the var keyword become the properties and methods of the window object.
- The window object exposes the functionality of the web browser to the webpage.

1.) alert():

- The browser can invoke a system dialog to display information to the user.
- To invoke an alert system dialog, you invoke the alert() method of the window object.
- The alert() is a method of the window object.
- The alert() method is modal and synchronous.
- Use the alert() method to display information that you want users to acknowledge.
- Syntax:

```
window.alert(message);
        OR
alert(message);
```

- The message is a string that contains information that you want to show to users.
- Example:

```
window.alert('Welcome to JavaScriptTutorial.net!');
        OR
alert('Welcome to Browser Object Model');
```

- When the alert() method is invoked, a system dialog shows the specified message to the user followed by a single OK button.
- Note : the alert dialog is synchronous and modal. It means that the code execution stops when a dialog is displayed and resumes after it has been dismissed.

2.) confirm():

- The confirm() is a method of the window object.
- The confirm() shows a system dialog that consists of a question and two buttons: OK and Cancel.
- The confirm() returns true if the OK button was clicked or false if the Cancel button was selected.
- Syntax:

```
let result = window.confirm(question);
```

- In this syntax:
  1. The question is an optional string to display in the dialog.
  2. The result is a Boolean value indicating whether the OK or Cancel button was clicked. If the OK button is clicked, the result is true; otherwise, the result is false.
- The confirmation dialog is modal and synchronous. It means that the code execution stops when a dialog is displayed and resumes after it has been dismissed.
- Example:

```
let result = confirm('Are you sure you want to delete?');
let message =result ? 'You clicked the OK button' :'You clicked the Cancel button';
alert(message);
```

3.) Window Size:

- The window object has four properties related to the size of the window.
- 1. innerWidth and innerHeight:

JAVASCRIPT

The innerWidth and innerHeight properties return the size of the page viewport inside the browser window (not including the borders and toolbars).

- 2. outerWidth and outerHeight:

  The outerWidth and outerHeight properties return the size of the browser window itself.

4.) Open a new window:

- To open a new window or tab, you use the window.open() method.
- Syntax:

```
window.open(url, windowName, [windowFeatures]);
```

- The window.open() method accepts three arguments:
  1. The URL to load
  2. The window target
  3. A string that represents the windowâ€™s features.
- The third argument (windowFeatures) is a comma-delimited string of settings, specifying displaying information for the new window such as width, height, menubar, and resizable.

5.) Resize a window:

- To resize a window you use the resizeTo() method of the window object.
- Syntax:

```
window.resizeTo(width,height);
```

- Example:

```
let jsWindow = window.open('./index.html','about','height=600,width=800');
setTimeout(() => {
  jsWindow.resizeTo(600, 300);
}, 3000);
```

6.) resizeBy():

- The window.resizeBy() method allows you to resize the current window by a specified amount:
- Syntax:

```
window.resizeBy(deltaX,deltaY);
```

- Example:

```
letjsWindow=window.open('http://localhost/js/about.html','about','height=
600,width=600');
        // shrink the window, or resize the window  to 500x500
setTimeout(() => {
 jsWindow.resizeBy(-100, -100);
}, 3000);
```

Location Object:

- The Location object represents the current URL of a page.
- It can be accessed via window.location or document.location.
- The Location object has a number of properties that represent the URL such as protocol, host, pathname, and search.
- The location object is a property of the window object.

Location Object Properties:

1. location.href:

- The location.href is a string that contains the entire URL.
- Example:http://localhost:8080/js/index.html?type=listing&page=2#title

2. Location.protocol:
   - The location.protocol represents the protocol scheme of the URL including the final colon (:).
   - Example: 'http:'
3. Location.host:
   - The location.host represents the hostname:
   - Example: "localhost:8080"
4. Location.port:
   - The location.port represents the port number of the URL.
   - Example: "8080"
5. Location.pathname:
   - The location.pathname contains an initial '/' followed by the path of the URL.
   - Example: "/js/index.html"
6. Location.search:
   - The location.search is a string that represents the query string of the URL:
   - Example: "?type=listing&page=2"
7. Location.hash:
   - The location.hash returns a string that contains a â€˜#â€™ followed by the fragment identifier of the URL.
   - Example: "#title"
8. Location.origin:
   - The location.origin is a string that contains the canonical form of the origin.
   - http://localhost:8080
9. Location.password:
   - THe location.password is a string that represents the password specified before the domain name.

JAVASCRIPT

DOM

DOM:(document Object Model)

- DOM stands for document object model.
- The web browser uses DOM to represent the HTML document internally. Additionally, it provides a set of functions and methods to modify the HTML document programmatically.
- These functions and methods are often called DOM Application Programming Interfaces or DOM API.
- Using DOM API in JavaScript, you can manipulate the HTML document effectively.

What is The Dom?

- The DOM is a representation of an HTML or XML document as a tree structure.
- Each element in the document is represented as a node within this tree.

Node:

- A node refers to any of the various parts that make up the structure of a document.
- Every element, attribute, and piece of text in a webpage is represented as a node.
- There are different types of nodes in the DOM.

Types of Nodes:

1. Document Node: Represents the entire document.
2. Element Nodes: Represent HTML tags (e.g., `<div>`, `<p>`).
3. Text Nodes: Contain the text within an element.
4. Attribute Nodes: Represent the attributes of an element (e.g., `class`, `id`).
5. Comment Nodes: Represent comments in the HTML (e.g., `<!-- Comment -->`).

Hierarchy in DOM:

- Nodes in the DOM are related in a hierarchical manner:
  1. Parent Node: A node that contains other nodes.
  2. Child Node: Nodes that are directly inside a parent node.
  3. Sibling Nodes: Nodes that share the same parent.

Below is a simple representation of how an HTML document is structured as a DOM tree:

```
<!DOCTYPE html>
    <html>
      <head>
        <title>Page Title</title>
      </head>
      <body>
        <div id="main">
          <p>Hello, World!</p>
```

```
    </div>
  </body>
</html>
```

DOM Tree Representation:

```
          Document
             |
           <html>
          /       \
      <head>      <body>
        |            |
   <title>        <div id="main">
        |                |
"Page Title"         <p>
                        |
              "Hello, World!"
```

Direct Access Method:

1. document.all:
- Returns a collection (similar to an array) of all elements in the document.
- Syntax:
  document.all
- Example:
  console.log(document.all[0]); // Logs the first element in the document.
2. document.scripts:
- Returns an HTMLCollections of all <script> elements in the documents.
- Synatx:
  document.scripts
- Example:
  console.log(document.scripts.length); // Logs the number of scripts in the document.
3. Document.images:
- Returns an HTMLCollections of all <img> elements in the document.
- Syntax:
  Document.images
- Example:

  console.log(document.images[0].src); // Logs the source of the first image.

4. document.links:
- Returns an HTMLCollection of all <a> elements with an href attribute in the document.
- syntax:
  document.links
- Example:
  console.log(document.links[0].href); // Logs the URL of the first
5. document.forms:
- Returns an HTMLCollection of all <form> elements in the document.
- Syntax:
  document.forms
6. document.body:

JAVASCRIPT

- Returns the <body> element of the document.
- Syntax:

```
document.body
```

- Example:

```
console.log(document.body.innerHTML); // Logs the entire content inside <body>.
```

Indirect Access Methods:

1. getElementById() :

- The document.getElementById() returns a DOM element specified by an id or null if no matching element is found.
- If multiple elements have the same id, even though it is invalid, the getElementById() returns the first element it encounters.
- Syntax:

```
const element = document.getElementById(id);
```

- In this syntax:

  id is a string that represents the id of the element to select.
- Note: the method matches ID case-sensitively. For example, the 'root' and 'Root' are different.
- If the document has no element with the specified id, the getElementById() method returns null.

2. getElementsByName():

- The getElementsByName() accepts a name which is the value of the name attribute of elements and returns a NodeList of elements.
- Every element on an HTML document may have a name attribute.
- The NodeList is an array-like object, not an array object.
- Syntax:

```
let elements = document.getElementsByName(name);
```

- Example:

```
<input type="radio" name="language" value="JavaScript">
<input type="radio" name="language" value="JavaScript">
let elements = document.getElementsByName(language);
```

3. getElementsByClassName():

- The getElementsByClassName() method returns an HTMLCollection of elements whose class names match one or more specified class names.
- Syntax:

```
getElementsByClassName(names).
```

- In this syntax:

  names represent one or more class names to match. If you use multiple class names, you need to separate them by a space
- Travesing Elements.
- The getElementsByClassName() method returns a HTMLCollection of the matched elements.
- If no element in the document matches the class names, the getElementsByClassName() method returns an empty HTMLCollection [].
- Example:

```
<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="UTF-8">
```

```
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>JavaScript getElementsByClassName</title>
</head>
<body>
 <header>
   <nav>
     <ul id="menu">
         <li class="item">HTML</li>
         <li class="item">CSS</li>
         <li class="item highlight">JavaScript</li>
         <li class="item">TypeScript</li>
     </ul>
   </nav>
   <h1>getElementsByClassName Demo</h1>
 </header>
 <section>
   <article>
     <h2 class="secondary">Example 1</h2>
   </article>
   <article>
     <h2 class="secondary">Example 2</h2>
   </article>
 </section>
 </body>
 </html>

   let menu = document.getElementById('menu');
   let items = menu.getElementsByClassName('item');
   console.log(items);
```

4. getElementsByTagName():

- The getElementsByTagName() is a method of the document object.
- The getElementsByTagName() method accepts a tag name such as h1, a, and img and returns a HTMLCollection of elements with the matching tag name.
- The HTMLCollection is an array-like object.
- Syntax:

```
let elements = document.getElementsByTagName(tagName);
```

5. querySelector():

- The querySelector() method allows you to select the first element that matches one or more CSS selectors.
- Syntax:
- let element = parentNode.querySelector(selector);
- In this syntax, the selector is a CSS selector or a group of CSS selectors to match the descendant elements of the parentNode.
- If no element matches the CSS selectors, the querySelector() returns null.
- The querySelector() method is available on the document object or any Element object.

6. querySelectorAll():

- the querySelectorAll() method to select all elements that match a CSS selector or a group of CSS selectors.

JAVASCRIPT

- Syntax:
- let elementLsist = parentNode.querySelectorAll(selector);
- The querySelectorAll() method returns a NodeList of elements that match the CSS selector.
- A CSS selector defines elements to which a CSS rule applies.

1. parentNode:
   - To get the parent node of a specified node in the DOM tree, you use the parentNode property:
   - let parent = node.parentNode;
   - The parentNode is read-only.
   - The Document and DocumentFragment nodes do not have a parent. Therefore, the parentNode will always be null.
   - If you create a new node but haven't attached it to the DOM tree, the parentNode of that node will also be null.
   - The node.parentNode returns the read-only parent node of a specified node or null if it does not exist.
   - Example:

```
<!DOCTYPE html>
<html>
<head>
<meta charset="utf-8">
<title>JavaScript parentNode</title>
</head>
<body>
  <div id="main">
    <p class="para">This is a note!</p>
  </div>
  <script>
    let note = document.querySelector('#para');
    console.log(note.parentNode);
  </script>
</body>
</html>
```

   - How it works:
     1. Select the element with the .note class by using the querySelector() method.
     2. Find the parent node of the element.

MCQ'S:

Q.1 What does the parentNode property return?

a. The parent element of the specified node

b. The first child element of the specified node

c. The last child element of the specified node

JAVASCRIPT

d. The sibling element of the specified node

ans: a

Q.2 Given the HTML Hello , how can you access the parent element of the paragraph?

a. document.getElementById('parent').parentNode

b. document.getElementById('child').parentNode

c. document.getElementById('child').childNode

d. document.getElementById('parent').childNode

ans: b

Q.3 What will document.body.parentNode return in a typical HTML document?

a. null

b. The <html> element

c. The <head> element

d. The <body> element

ans: b

Q.4 If an element does not have a parent node, what will the parentNode property return?

a. null

b. undefined

c. The element itself

d. An error

ans: a

Q.5 How can you check if an element has a parent node in JavaScript?

a. if (element.parentNode != null)

b. if (element.parentNode == element)

c. if (element.childNodes.length > 0)

JAVASCRIPT

d. if (element.parentNode == false)

2. Siblings of an Element:

1. nextElementSibling:

- To get the next sibling of an element, you use the nextElementSibling
- let nextSibling = currentNode.nextElementSibling;
- The nextElementSibling returns null if the specified element is the last one in the list.
- Example:

```
<ul id="menu">
    <li>Home</li>
    <li>Products</li>
    <li class="current">Customer Support</li>
    <li>Careers</li>
    <li>Investors</li>
    <li>News</li>
    <li>About Us</li>
</ul>
let current = document.querySelector('.current');
let nextSibling = current.nextElementSibling;

console.log(nextSibling);
```

- How it works:
  1.select the list item whose class is current using selecting method.
  2.get the next sibling of that list item using the nextElementSibling property.

- Q. How to get all the next siblings of an element:

```
let current = document.querySelector('.current');
let nextSibling = current.nextElementSibling;
while(nextSibling) {
    console.log(nextSibling);
    nextSibling = nextSibling.nextElementSibling;
}
```

2. nextSibling:

- Returns the next sibling node of any type, including text nodes, comment nodes, and element nodes.
- If there are no sibling nodes after the specified one (like when it's the last one), it returns null.
- Example:

```
<div id="first">First</div>
    <!-- This is a comment -->
    <div id="second">Second</div>
    <script>
      const firstDiv = document.getElementById('first');
      console.log(firstDiv.nextSibling);
    </script>
```

3. previousElementSibling:

- To get the previous siblings of an element, you use the previousElementSibling.
- let current = document.querySelector('.current');
  let prevSibling = currentNode.previousElementSibling;
- The previousElementSibling property returns null if the current element is the first one in the list.

4. Previoussibilings:

- The previousSibling property returns the previous sibling node of the specified node, which could be any type of node (text, comment, element, etc.).
- If there is no previous sibling node, it returns null.
- Example:

```
<div id="first">First</div>
    <!-- This is a comment -->
<div id="second">Second</div>
<script>
    const secondDiv = document.getElementById('second');
    console.log(secondDiv.previousSibling); // Logs the comment node
</script>
```

3 Child Elements of a node:

1. firstChild:
   - To get the first child element of a specified element, you use the firstChild
   - If the parentElement does not have any child element, the firstChild returns null.
   - The firstChild property returns a child node which can be any node type such as an element node, a text node, or a comment node.
   - Example:

```
<!DOCTYPE html>
<html>
<head>
<meta charset="utf-8">
<title>JS Get Child Elements</title>
</head>
<body>
<ul id="menu">
<li class="first">Home</li>
<li>Products</li>
<li class="current">Customer Support</li>
<li>Careers</li>
<li>Investors</li>
<li>News</li>
<li class="last">About Us</li>
</ul>
</body>
</html>
```

   - The following script shows the first child of the #menu element:

```
let content = document.getElementById('menu');
let firstChild = content.firstChild.nodeName;
console.log(firstChild);
```

2. firstElementChild:

JAVASCRIPT

- to get the first child with the Element node only.
- let firstElementChild = parentElement.firstElementChild;
- Example:

```
<!DOCTYPE html>
<html>
 <head>
  <meta charset="utf-8">
  <title>JS Get Child Elements</title>
 </head>
<body>
  <ul id="menu">
   <li class="first">Home</li>
   <li>Products</li>
   <li class="current">Customer Support</li>
   <li>Careers</li>
   <li>Investors</li>
   <li>News</li>
   <li class="last">About Us</li>
  </ul>
</body>
</html>
```

- The following code returns the first list item which is the first child element of the.

```
let content = document.getElementById('menu');
console.log(content.firstElementChild);  //   <li class="first">Home</li>
```

- How it works:

  In this example:

  1. select the #menu element by using the getElementById() method.
  2. get the first child element by using the firstElementChild property.

3. lastChild:

- To get the last child element of a node, you use the lastChild property:

  let lastChild = parentElement.lastChild;
- In case the parentElement does not have any child element, the lastChild returns null.
- The lastChild property returns the last element node, text node, or comment node.
- Note: If you want to select only the last child element with the element node type, you use the lastElementChild property:

4. lastElementChild():

- If you want to select only the last child element with the element node.
- let lastChild = parentElement.lastElementChild;
- Example:

```
<!DOCTYPE html>
<html>
 <head>
  <meta charset="utf-8">
  <title>JS Get Child Elements</title>
```

```
</head>
<body>
 <ul id="menu">
  <li class="first">Home</li>
  <li>Products</li>
  <li class="current">Customer Support</li>
  <li>Careers</li>
  <li>Investors</li>
  <li>News</li>
  <li class="last">About Us</li>
 </ul>
</body>
</html>
```

- The following code returns the list item which is the last child element of the menu.

```
Let menu = document.getElementById('menu');
Consoloe.log(main.lastChildElementChild); //<li class = "last"> About us</li>
```

Manipulating Elements Methods:

1. textContent():
   - To get the text content of a node and its descendants, you use the textContent.
   - The textcontent will give content how you writing in your html document.
   - Syntax:

     let text = node.textContent;
   - Example:

```
<div id="note">
    JavaScript textContent Demo!
    <span style="display:none">Hidden Text!</span>
    <!-- my comment -->
</div>
```

   - The following example uses the textContent property to get the text of the element.

```
let note = document.getElementById('note');
console.log(note.textContent);
```

     output:

```
JavaScript textContent Demo!
Hidden Text!
```

   - How it works:
     1. First, select the div element with the id note by using the getElementById() method.
     2. Then, display the text of the node by accessing the textContent property.
2. innerText():
   - innerText retrieves or sets the visible text content of an element, excluding any hidden elements.
   - Syntax: let textContent = element.innerText;
   - Example

```
<div id="note">
```

```
JavaScript textContent Demo!
<span style="display:none">Hidden Text!</span>
<!-- my comment -->
</div>

let note = document.getElementById('note');
console.log(note.innerText);
```

output:

```
JavaScript textContent Demo!
```

3. innerHtml():
   - innerHTML retrieves or sets the HTML content inside an element, including text and nested HTML tags.
   - whenever we need content along with the html element that time we are using this innerHtml.
   - Syntax:

     To get the HTML content:

     ```
     let htmlContent = element.innerHTML;
     ```

     To set the HTML content:

     ```
     element.innerHTML = "<p>New Content</p>";
     ```

   - Example:

     Consider the following HTML structure:

     ```
     <div id="example">
       <p>Original Text</p>
     </div>

     let element = document.getElementById("example");
     console.log(element.innerHTML); // Outputs: "<p>Original Text</p>"
     element.innerHTML = "<span>Updated Text</span>";
     ```

   - the content of the will change to: Updated Text.
4. CreateElement():
   - the document.createElement() to create a new HTML element and attach it to the DOM tree.
   - The document.createElement() accepts an HTML tag name and returns a new Node with the Element type.
   - Syntax:

     ```
     let element = document.createElement(htmlTag);
     ```

   - Example:
     1. Creating a new div example

        ```
        let div = document.createElement('div');
        ```

     2. add the content in the element.

        ```
        div.innertext = "CreateElement example".
        ```

5. appendChild():
   - The appendChild() method allows you to add a node to the end of the list of child nodes of a specified parent node.
   - Syntax:

     ```
     parentNode.appendChild(childNode);
     ```

JAVASCRIPT

- In this method, the childNode is the node to append to the given parent node. The appendChild() returns the appended child.
- Example:

```
<ul id="menu"> </ul>

const menu = document.querySelector('#menu');
let li = document.createElement('li');
li.textContent = "Home";
menu.appendChild(li)
```

6. Attribute methods:

1.) setAttribute():

- To set the value of an attribute on a specified element, you use the setAttribute() method
- Syntax:

```
element.setAttribute(name, value);
```

- In the syntax,
  1. The name specifies the attribute name whose value is set.
  2. The value specifies the value to assign to the attribute.
- The setAttribute() returns undefined.
- Note: if the attribute already exists on the element, the setAttribute() method updates the value.
- Example:

```
<button id="btnSend">Send</button>
let btnSend = document.querySelector('#btnSend');
btnSend.setAttribute('name', 'send');
btnSend.setAttribute('disabled', '');
```

1.) Select the button with the id btnSend by using the querySelector() method.
2.) Set the value of the name attribute to send using the setAttribute() method.
3.) Set the value of the disabled attribute so that when users click the button, it will do nothing.

2.) getAttribute():

- To get the value of an attribute on a specified element, you call the getAttribute() method of the element.
- Syntax:

```
let value = element.getAttribute(name);
```

- The getAttribute() accepts an argument which is the name of the attribute from which you want to return the value.
- If the attribute exists on the element, the getAttribute() returns a string that represents the value of the attribute.
- In case the attribute does not exist, the getAttribute() returns null.
- Example:

```
<a href="https://www.flipkart.com/"
    target="_blank"
    id="js">
  flipkart
</a>

let link = document.querySelector('#js');
```

JAVASCRIPT

```
let target = link.getAttribute('target');
console.log(target);
output:
    _blank
```

3.) removeAttribute():

- The removeAttribute() method removes an attribute with a specified name from an element.
- Syntax:

```
element.removeAttribute(name);
```

- The removeAttribute() accepts an argument which is the name of the attribute you want to remove.
- If the attribute does not exist, the removeAttribute() method will not raise an error.
- The removeAttribute() returns a value of undefined.
- Example:

```
<a href="https://www.flipkart.com/"
    target="_blank"
    id="js">flipkart
</a>
let link = document.querySelector('#js');
link.removeAttribute('target');
```

- How it works:
  1. Select the link element with the id using the querySelector() method.
  2. Remove the target attribute of the link by calling the removeAttribute() on the selected link element.

4.) has Attribute():

- To check whether an element has a specified attribute or not, you use the hasAttribute() method.
- Syntax:

```
let result = element.hasAttribute(name);
```

- In this syntax:

  name specifies the attribute name you want to check in the element.
- The hasAttribute() returns true if the element contains the specified attribute or false otherwise.
- Example:

```
<button id="btnSend" disabled>Send</button>
let disabled = btn.hasAttribute('disabled');
console.log(disabled);
```

CSS Using DOM:

1. Using Style Property
   - To set the inline style of an element, you use the style property of that element.
   - Syntax:

```
element.style
```

   - The style property returns the read-only CSSStyleDeclaration object that contains a list of CSS properties.
   - Example:

JAVASCRIPT

```
element.style.color = 'red';
```

- If the CSS property contains hyphens (-) you can use the array-like notation ([]) to access the property:
- Example:

```
element.style.['-webkit-text-stock'] = 'unset';
```

2. setAttribute():
   - Using setAttribute() methods also we can add the css.
   - set the style attribute along with css property and values.
   - Syntax:
   - setAttribute("style","key1:value1;key2:value2;")
   - Example:

```
element.setAttribute('style','color:red;background-color:yellow');
```

classList:

- The classList is a read-only property of an element that returns a live collection of CSS classes.
- Syntax:

```
const classes = element.classList;
```

- Even though the classList is read-only, but you can manipulate the classes it contains using various methods.
- Manipulating CSS classes of the element via the classList.

1. Get the CSS classes of an element:
   - Suppose that you have a div element with two classes: main and red.

```
<div id="content" class="main red">JavaScript classList</div>

let div = document.querySelector('#content');
for (let cssClass of div.classList) {
console.log(cssClass);
}
```

   - First, select the div element with the id content using the querySelector() method.
   - Then, iterate over the elements of the classList and show the classes in the Console window.
2. add() method of the classList:
   - To add one or more CSS classes to the class list of an element, you use the add() method of the classList.
   - syntax:

```
element.classList.add("class")
```

   - the following code adds the info class to the class list of the div element with the id content.

```
<div id="content" class="main red">JavaScript classList</div>
let div = document.querySelector('#content');
div.classList.add('info');
```

   - The following example adds multiple CSS classes to the class list of an element.

```
<div id="content" class="main red">JavaScript classList</div>
let div = document.querySelector('#content');
div.classList.add('info','visible','block');
```

3. remove() method:

- To remove a CSS class from the class list of an element, you use the remove() method.
- Syntax:

```
element.classList.remove("classes")
```

- Example:

```
<div id="content" class="main red">JavaScript classList</div>
let div = document.querySelector('#content');
div.classList.add('info','visible','block');
div.classList.remove('visible');
```

- Like the add() method, you can remove multiple classes once.

4. replace() method:
   - To replace an existing CSS class with a new one, you use the replace() method.
   - syntax:

```
element.classList.replace("oldClass","newClass")
```

   - Example:

```
let div = document.querySelector('#content');
div.classList.replace('info','warning');
```

5. contains() method:
   - To check if the element has a specified class, you use the contains() method:
   - The contains() method returns true if the classList contains a specified class otherwise false
   - syntax:
   - element.classList.contains("class")
   - Example:

```
let div = document.querySelector('#content');
div.classList.contains('warning'); // true
```

6. toggle() method:
   - If the class list of an element contains a specified class name, the toggle() method removes it.
   - If the class list doesn't contain the class name, the toggle() method adds it to the class list.
   - syntax:

```
element.classList.toggle("className")
```

   - Example:

```
let div = document.querySelector('#content');
div.classList.toggle('visible');
```

Short Revision of classList:

1. The element's classList property returns the live collection of CSS classes of the element.
2. Use the add() and remove() methods to add CSS classes to and remove CSS classes from the class list of an element.
3. Use the replace() method to replace an existing class with a new one.
4. Use the contains() method to check if the class list of an element contains a specified class.
5. Use the toggle() method to toggle a class.

JAVASCRIPT

Events

Event:

- Event is an action or occurrence when that happens in the browser.
- Event is an object.
- An event may have an event handler, a function that runs when the event occurs.
- An event handler, also known as an event listener, listens for the event and executes when it happens

Event Handling:

- Event handling refers to the process of writing code to detect and respond to events triggered by the user or the browser.
- Event handling is achieved by attaching event listeners to elements in the DOM.
- These listeners "listen" for specific events and execute a predefined function, called an event handler, when the event occurs.

Steps in Event Handling:

1. Identify the type of event you want to handle (e.g., click, keypress, submit).
2. Attach an event listener: Use JavaScript to link the event to a specific function (the event handler).
3. Respond to the event: Define what should happen when the event occurs.

Types of Event Handlers:

1. Inline Event Handlers:
   - Defined directly in HTML using attributes.
   - Event handlers typically have names that begin with on, for example, the event handler for the click event is onclick.
   - syntax:
   - <element event="JavaScript code"></element>
   - Example:

```
1: <button onclick="alert('Button Clicked!')">Click Me</button>
2:
   <input type="button" value="Save" onclick="handleClick()">
   function handleClick() {
        alert('Clicked!');
   }
```

   - In this example, the handleClick() function is executed when the button is clicked.
   - Disadvantages:
     1. Inline code mixes JavaScript with HTML (not recommended).
     2. Harder to manage for complex projects.

2. DOM Property Handlers:

   - Assign a handler function directly to an element's event property.
   - Each element has event handler properties such as onclick
   - Syntax:

```
element.eventType = functionNameOrCode;
```


JAVASCRIPT

- In this syntax
  1.element: The DOM element you want to attach the handler to.
  2.eventType: The type of event (e.g., click, change).
  3.functionNameOrCode: A function or inline code to execute when the event is triggered.
- the this value inside the event handler, you can access the element's properties and methods.
- Example:

```
const button = document.getElementById('myButton');
button.onclick = () => {
  alert('Button clicked!');
};
```

- Only one handler can be assigned to an event type at a time. Assigning a new handler overwrites the previous one.

```
button.onclick = () => alert('First Handler');
button.onclick = () => alert('Second Handler'); // Overwrites the first
```

3. By using addEventListener() Method:

- To define a function that will be executed when the button is clicked, you need to register an event handler using the addEventListener() method.
- The addEventListener() method accepts three arguments: an event name, an event handler function, and a Boolean value that instructs the method to call the event handler during the capture phase (true) or during the bubble phase (false).
- Syntax:

```
element.addEventListener(event, function, useCapture);
```

- In this Syntax
  1.event: A string representing the name of the event (e.g., "click", "mouseover", "keydown", etc.).
  2.function: The event handler function to execute when the event occurs. This can also be an anonymous function or an arrow function.
  3.useCapture (optional): A boolean indicating whether the event should be captured during the capturing phase (true) or the bubbling phase (false).
- Defaults to false.
- Example:

```
let btn = document.querySelector('#btn');
 btn.addEventListener('click', function(event) {
alert(event.type); // click
});
```

- It is possible to add multiple event handlers to handle a single event.
- Example:

```
let btn = document.querySelector('#btn');
btn.addEventListener('click',function(event) {
      alert(event.type); // click
  });
 btn.addEventListener('click',function(event) {
       alert('Clicked!');
  });
```

JAVASCRIPT

4. removerEventListener():

- The removeEventListener() removes an event listener that was added via the addEventListener().you need to pass the same arguments as were passed to the addEventListener().
- Syntax:
- element.removeEventListener(event, listener, options);
- In this syntax:
  1.event: The name of the event to remove, such as "click", "keydown", or "resize".
  2.listener: The event handler function that was previously added with addEventListener. This must be the exact same function reference.
  3.options (Optional): An object or boolean indicating options such as capture. It must match the options used in addEventListener
- Example:

```
let btn = document.querySelector('#btn');
// add the event listener
let handleClick = function() {
  alert('Clicked!');
};
btn.addEventListener('click', handleClick);
// remove the event listener
btn.removeEventListener('click', handleClick);
```

Event Propagation

- Event Propagation determines in which order the elements receive the event.
- Propagation refers to how events travel through the Document Object Model (DOM) tree
- Bubbling and Capturing are the two phases of propagation.

Event bubbling:

- In the event bubbling model, an event starts at the most specific element and then flows upward toward the least specific element (the document or even window).
- bubbling travels from the target element to the root.
- The target is the DOM node on which you click, or trigger with any other event.
- By default, most events use the bubbling phase when you add an event listener without specifying the third argument:
- For example, a button with a click event would be the event target. The root is the highest-level parent of the target. This is usually the document, which is a parent of the, which is a (possibly distant) parent of your target element.

JAVASCRIPT

```
Document  <---|
              |
              <--ElementHtml <--|
                                |
                                <-- ElementBody <--|
                                                   |
                                                   <--ElementDiv <--|
                                                                    |
                                                                    <--EventButton
```

-
  <button id="child">Click Me!</button>

```
let ElementDiv = document.getElementById("parent");
let EventButton =  document.getElementById("child");
ElementDiv .addEventListener("click", function() {
  alert("Parent Div Clicked!");
});
EventButton.addEventListener("click", function(event) {
      alert("Button Clicked!")
});
```

Event Capturing:

- It is the opposite of bubbling. The event handler is first on its parent component and then on the component where it was actually wanted to fire that event handler.
- In short, it means that the event is first captured by the outermost element and propagated to the inner elements.
- Capturing travels from the root to the target.

```
Document  --->|
              |
              --> ElementHtml -->|
                                 |
                                 --> ElementBody -->|
                                                    |
                                                    --> ElementDiv -->|
                                                                      |
                                                                      -->EventButton
```

- Example:

  <button id="child">Click Me!</button>

```
let ElementDiv = document.getElementById("parent");
let EventButton =  document.getElementById("child");
ElementDiv .addEventListener("click", function() {
  alert("Parent Div Clicked!");
},{ capture: true });
EventButton.addEventListener("click", function(event) {
      alert("Button Clicked!")
},{ capture: true });
```

addEventListener():

- addEventListener() has an optional third parameter - which takes its argument as a boolean - which controls the phase of the propagation.
- The parameter is called Capture, and passing true will cause the listener to be on the capturing phase. The default is false, which will apply it to the bubbling phase.

stopPropagation():

- It will prevent further propagation through the DOM tree, and only run the event handler from which it was called.
- Example:

```
function first() {
    console.log(1);
  }
function second() {
    console.log(2);
  }
var button = document.getElementById("button");
var container = document.getElementById("container");
button.addEventListener("click", first);
container.addEventListener("click", second);
```

- In the above example, clicking the button will cause the console to print 1, 2. If we wanted to modify this so that only the button's click
- Event is triggered, we could use Event.stopPropagation() to immediately stop the event from bubbling to its parent.

```
function first(event) {
  event.stopPropagation();
  console.log(1);
}
```

- This modification will allow the console to print 1, but it will end the event chain right away, preventing it from reaching 2.

preventDefault():

- To prevent the default behavior of an event, you use the preventDefault() method.
- For example, when you click a link, the browser navigates you to the URL specified in the href attribute.
- <a href="https://www.flipkart.com/">FlipKart</a>
- You can prevent this behavior by using the preventDefault() method of the event object.

```
let link = document.querySelector('a');
link.addEventListener('click',function(event) {
    console.log('clicked');
    event.preventDefault();
});
```

Event Delegation:

JAVASCRIPT

- Event delegation is a technique in JavaScript where a single event listener is attached to a parent element instead of attaching event listeners to multiple child elements.
- Event delegation is a technique that relies on event bubbling to manage events efficiently.
- For example:

```
<ul id="menu">
    <li><a id="home">home</a></li>
    <li><a id="dashboard">Dashboard</a></li>
    <li><a id="report">report</a></li>
</ul>
```

- To handle the click event of each menu item, you may add the corresponding click event handlers:

```
let home = document.querySelector('#home');
home.addEventListener('click',(event) => {
console.log('Home menu item was clicked');
});
let dashboard = document.querySelector('#dashboard');
dashboard.addEventListener('click',(event) => {
console.log('Dashboard menu item was clicked');
});
let report = document.querySelector('#report');
report.addEventListener('click',(event) => {
console.log('Report menu item was clicked');
});
```

- Instead of having multiple event handlers, you can assign a single event handler to handle all the click events:

```
let menu = document.querySelector('#menu');
menu.addEventListener('click', (event) => {
let target = event.target;
switch(target.id) {
  case 'home':
    console.log('Home menu item was clicked');
    break;
  case 'dashboard':
    console.log('Dashboard menu item was clicked');
    break;
  case 'report':
    console.log('Report menu item was clicked');
    break;
 }
});
```

- In JavaScript, if you have a large number of event handlers on a page, these event handlers will directly impact the
- Performance because of the following reasons:

1. Each event handler is a function which is also an object that takes up memory. The more objects in the memory, the slower the performance.
2. It takes time to assign all the event handlers, which causes a delay in the interactivity of the page.

JAVASCRIPT

Event Types:

1. Mouse Events
2. Keyboard Events

1. Mouse Events:
   - Mouse events fire when you use the mouse to interact with the elements on the page.
   - mousedown, mouseup, and click events:
   - When you click an element, there are no less than three mouse events fire in the following sequence:
   - The mousedown fires when you press the mouse button on the element.
   - The mouseup fires when you release the mouse button on the element.
   - The click fires when one mousedown and one mouseup detected on the element.

   dbclick event:

   - The dblclick event fires when you double-click over an element.

   mousemove:

   - The mousemove event fires repeatedly whenever you move the mouse cursor around an element.
   - This mousemove event fires many times per second as the mouse is moved around, even if it is just by one pixel.

   mouseout:

   - The mouseout fires when the mouse cursor is over an element and then moves another element.

   mouseenter:

   - The mouseenter fires when the mouse cursor is outside of an element and then moves inside the boundaries of the element.

   mouseleave:

   - The mouseleave fires when the mouse cursor is over an element and then moves to the outside of the element's boundaries.

2. Keyboard Events:
   - When you interact with the keyboard, the keyboard events are fired.

   keydown:

   - fires when you press a key on the keyboard and fires repeatedly while you're holding down the key.
   - Example:

```
    <style>
  body {
```

JAVASCRIPT

```
    font-family: Arial, sans-serif;
    margin: 0;
    height: 100vh;
    display: flex;
    justify-content: center;
    align-items: center;
    position: relative;
    background-color: #f0f0f0;
  }
  #box {
    width: 50px;
    height: 50px;
    background-color: red;
    position: absolute;
    top: 100px;
    left: 100px;
  }
   </style>
   </head>
 <body>
<div id="box"></div>

<script>
  const box = document.getElementById('box');
  let topPosition = 100;
  let leftPosition = 100;
  document.addEventListener('keydown', (event) => {
    const step = 10;

    switch (event.key) {
      case 'ArrowUp':
        topPosition -= step;
        break;
      case 'ArrowDown':
        topPosition += step;
        break;
      case 'ArrowLeft':
        leftPosition -= step;
        break;
      case 'ArrowRight':
        leftPosition += step;
        break;
    }

    box.style.top = `${topPosition}px`;
    box.style.left = `${leftPosition}px`;
  });
</script>
```

JAVASCRIPT

```
</body>
</html>
```

keyup:

- Fires when you release a key on the keyboard.
- The 'keyup' event in JavaScript is triggered when the user releases a key on the keyboard.
- It is commonly used for tasks that require real-time updates based on user input, such as form validation, search suggestions, or live character counting.
- Example:

```
<input type="text" id="textInput" placeholder="Type something here..." />
<p>Character Count: <span id="charCount">0</span></p>

const textInput = document.getElementById('textInput');
const charCount = document.getElementById('charCount');

textInput.addEventListener('keyup', () => {
  charCount.textContent = textInput.value.length;
});
```

keypress:

- Fires when you press a character keyboard like a,b, or c, not the left arrow key, home, or end keyboard.
- The keypress also fires repeatedly while you hold down the key on the keyboard.
- Example:

```
<h1>Press any key to see the output in the console!</h1>
<script>
  document.addEventListener("keydown", function (event) {
  console.log(`Key pressed: ${event.key}`);
  });
</script>
```

JAVASCRIPT

Es6 Concepts

Rest Parameter:

- ES6 provides a new kind of parameter so-called rest parameter that has a prefix of three dots (...).
- A rest parameter allows you to represent an indefinite number of arguments as an array.
- Example:

```
function fn(a,b,...args) {
    console.log(args);
}
```

- The last parameter (args) is prefixed with the three dots (...). It's called a rest parameter (...args).
- All the arguments you pass to the function will map to the parameter list. In the syntax above, the first argument maps to a, the second one maps to b, and the third, the fourth, etc., will be stored in the rest parameter args as an array.

```
fn(1, 2, 3, "A", "B", "C");
```

- The args array stores the following values:

```
[3,'A','B','C']
```

- If you pass only the first two parameters, the rest parameter will be an empty array:

```
fn(1,2);
```

- The args will be [].

Spread operator:

- ES6 provides a new operator called spread operator that consists of three dots (...).
- The spread operator allows you to spread out elements of an object.
- The spread operator is denoted by three dots (...).
- The spread operator can be used to clone an iterable object or merge iterable objects into one.
- Example:

```
const odd = [1,3,5];
const combined = [2,4,6, ...odd];
console.log(combined); //[ 2, 4, 6, 1, 3, 5 ]
```

- In this example, the three dots (...) located in front of the odd array is the spread operator. The spread operator (...) unpacks
- The elements of the odd array.

JavaScript spread operator and array:

1.) Constructing array literal:
   - The spread operator allows you to insert another array into the initialized array when you construct an array using the literal form.
   - Example:

```
let initialChars = ['A', 'B'];
let chars = [...initialChars, 'C', 'D'];
console.log(chars); // ["A", "B", "C", "D"]
```

2.) Concatenating arrays:

JAVASCRIPT

- You can use the spread operator to concatenate two or more arrays.
- Example:

```
let numbers = [1, 2];
let moreNumbers = [3, 4];
let allNumbers = [...numbers, ...moreNumbers];
console.log(allNumbers); // [1, 2, 3, 4]
```

3.) Copying an array:
- In addition, you can copy an array instance by using the spread operator:
- Example:

```
let scores = [80, 70, 90];
let copiedScores = [...scores];
console.log(copiedScores); // [80, 70, 90]
```

JavaScript spread operator and object:

1.) Cloning an Object:
- The spread operator creates a shallow copy of an object.
- Example:

```
const obj1 = { a: 1, b: 2 };
const newObj = { ...obj1 };
console.log(newObj); // { a: 1, b: 2 }
```

2.) Merging Object:
- Combine multiple objects into one.
- Example:

```
const obj1 = { a: 1, b: 2 };
const obj2 = { b: 3, c: 4 };
const mergedObj = { ...obj1, ...obj2 };
console.log(mergedObj); // { a: 1, b: 3, c: 4 }
```

- Note:In case of overlapping keys, the properties from the later objects overwrite the earlier ones.

Modules

- In JavaScript, a module is a file that contains code that can be imported into other code files.
- With the help of modules, developers can create more modular and scalable applications, improving overall code quality.
- The import and export keywords serve as the bridge that connects different modules.
- The export keyword makes variables or functions available to other modules.
- The import keyword is utilized for importing variables or functions into other modules.

Types of Modules:

1. Common Js Modules
2. Es6 Modules

1.) Common Js Modules:
- CommonJS modules are the original way to package JavaScript code for Node.js.

2.) Es6 Modules:

JAVASCRIPT

- ECMAScript modules are the official standard format to package JavaScript code for reuse.
- Es6 modules are again divided into two types.
  1. Named export Modules
  2. Export default module

  1. Named Export Modules:
     - Allows you to export multiple things by name, and you must import using the same name.
     - Must Match Names: The names of the exports must match exactly when importing.
     - When you import one or more of these named exports, you need to use {} to indicate which specific named export(s) .
     - Multiple Exports: You can export and import multiple items from the same module.
     - As Keyword: You can alias named exports using as.
     - Example:

```
Ex:1 // file: demo.js
     export const add = (a, b) => a + b;
     export const subtract = (a, b) => a - b;
     export const multiply = (a, b) => a * b;

     // file: app.js
     import { add, subtract } from './demo';
     console.log(add(2, 3));
     console.log(subtract(5, 2));
```

     - Combining Imports: If you want, you can import multiple named exports in one statement.
     - Ex:2

```
// file: demo.js
const name="Raj"
const age=27
function sayHello(){
    console.log("Hello");
}
export {name,age,sayHello}

// file: app.js
import {name, isSuperman} from './demo'
```

     - Note : you can import all named exports at once using the asterisk (*) .

```
import * as newlyImport from './demo'
```

  2. Export Default Module:
     - Allows you to export one default item per module, and the importing module can name it anything.
     - You have to export default keywords.
     - Example:

```
// file: demo.js
    export default function divide(a, b) {
        return a / b;
```

```
}

// file: app.js
import divide from './demo';
console.log(divide(10, 2)); // Output: 5
```

JAVASCRIPT